

CS2220: Introduction to Computational Biology

Sequence Alignment

Dynamic Programming



Somayyeh Koochi

Fall 2024



Adapted with modifications from lecture notes prepared by Phillip Compeau,

Bioinformatics Algorithms: An Active Learning Approach

Outline



- **Introduction to Sequence Alignment**
- The Manhattan Tourist Problem
- Sequence Alignment is the Manhattan Tourist Problem in Disguise
- An Introduction to Dynamic Programming: The Manhattan Tourist Problem Revisited
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment

Sequence Alignment



They tell us about

- ❧ Function or activity of a new gene/protein
- ❧ Structure or shape of a new protein
- ❧ Location or preferred location of a protein
- ❧ Stability of a gene or protein
- ❧ Origin of a gene or protein
- ❧ Origin or phylogeny of an organism
- ❧ And more...

The biological problem of sequence alignment



DNA-sequence-1

tcct**ct**gc**ct**tctgccatcat---caaccccaaagt

||||| ||| ||||| ||||| ||||| ||||| ||||| |||||

tcct**gt**gc**a**tctgcaatcat**ggg**caaccccaaagt

DNA-sequence-2

Alignment

The sequences are padded with gaps (dashes) so that wherever possible, columns contain identical characters from the sequences involved

Causes for sequence (dis)similarity



- mutation:** a nucleotide at a certain location is replaced by *another* nucleotide (e.g.: A**T**A → A**G**A)
- insertion:** at a certain location one new nucleotide is inserted inbetween two existing nucleotides (e.g.: AA → A**G**A)
- deletion:** at a certain location one existing nucleotide is deleted (e.g.: AC**T**G → AC**-**G)
- indel:** an **insertion** or a **deletion**

Comparing Genes

Comparing Genes Problem: *Compare two similar genes.*

- **Input:** Two genes.
- **Output:** How “similar” these genes are.

STOP and Think: How can we make this into a well-defined computational problem?

Hamming Distance

Hamming Distance Problem: *Compute the number of mismatching symbols between two strings.*

- **Input:** Two strings.
- **Output:** The number of “mismatched” symbols in the two strings.

ATGCATGC
TGCATGCA

Hamming distance = 8

Toward a More Accurate Problem



A**TGCATGC**–
–**TGCATGC**A

“Sliding” the strings reveals other similarities.

ATGCTTA
TGCATTAA

Furthermore, these strings have different lengths, so how should we compare them?

Toward a More Accurate Problem



Symbol Matching Problem: *Match as many symbols as possible between two strings.*

- **Input:** Two strings.
- **Output:** The greatest number of matched symbols in any “alignment” of the two strings.

A **TGC** – **TTA** –
– **TGC** A **TTA** A

Sequence Alignment as a Game



✧ At each turn, you have two choices

1. You can remove the first symbol from each sequence, in which case you earn a point if the symbols match;
2. Alternatively, you can remove the first symbol from either of the two sequences, in which case you earn no points but may set yourself up to earn more points in later moves.

❖ Your goal is to maximize the number of points.

Sequence Alignment as a Game

Growing alignment

Remaining symbols

Score

A
A

A T G T T A T A
A T C G T C C

T G T T A T A
T C G T C C

+1

A T
A T

G T T A T A
C G T C C

+1

A T -
A T C

G T T A T A
G T C C

A T - G
A T C G

T T A T A
T C C

+1

A T - G T
A T C G T

T A T A
C C

+1

A T - G T T
A T C G T -

A T A
C C

A T - G T T A
A T C G T - C

T A
C

A T - G T T A T
A T C G T - C -

A
C

A T - G T T A T A
A T C G T - C - C

From a Game to an Alignment



Given two strings v and w , an **alignment** of v and w is a two-row matrix such that:

- the first row contains symbols of v
- the second row contains symbols of w
- each row may also contain **gap symbols** (“-”)

```
A T - G T T A T A
A T C G T - C - C
```


From a Game to an Alignment



Given two strings v and w , an **alignment** of v and w is a two-row matrix such that:

- the first row contains symbols of v
- the second row contains symbols of w
- each row may also contain **gap symbols** (“-”)

A T - G T T A T A

A T C G T - C - C

Matches

From a Game to an Alignment



Given two strings v and w , an **alignment** of v and w is a two-row matrix such that:

- the first row contains symbols of v
- the second row contains symbols of w
- each row may also contain **gap symbols** (“-”)

A T - G T T **A** T **A**

A T C G T - **C** - **C**

Mismatches

From a Game to an Alignment



Given two strings v and w , an **alignment** of v and w is a two-row matrix such that:

- the first row contains symbols of v
- the second row contains symbols of w
- each row may also contain **gap symbols** (“-”)

A	T	-	G	T	T	A	T	A
A	T	C	G	T	-	C	-	C

Insertions

From a Game to an Alignment



Given two strings v and w , an **alignment** of v and w is a two-row matrix such that:

- the first row contains symbols of v
- the second row contains symbols of w
- each row may also contain **gap symbols** (“-”)

A	T	-	G	T	T	A	T	A
A	T	C	G	T	-	C	-	C

Deletions

Longest Common Subsequence



A **common subsequence** of v and w is a sequence of symbols occurring (not necessarily contiguously) in both v and w .

The **matches** in an alignment of v and w form a common subsequence of v and w .

A T – **G T** T A T A

A T C **G T** – C – C

DNA Alignment

- ❧ Aligning these three sequences from humans, chimpanzees, and macaques allows us to determine which positions in the sequence should be compared with one another.
- ❧ In the unaligned sequences, 15 spots are identical; in the aligned sequences, 22 spots are identical.³

Unaligned sequences

Human	A	C	A	T	T	A	T	G	G	A	C	A	G	G	T	A	A	G	T	A	A	A	A	A	C	A	T	A	T	T	
Chimpanzee	A	C	A	T	T	A	T	G	G	A	C	A	G	G	T	A	A	G	T	A	A	A	A	A	C	A	T	A	T	T	
Macaque	A	T	A	T	A	C	A	T	T	A	C	G	G	A	C	A	G	G	T	A	A	G	T	A	A	A	A	A	C	A	T

Aligned sequences

Human	A	C	A		T	T	A	T	G	G	A	C	A	G	G	T	A	A	G	T	A	A	A	A	A	A	C	A	T	A	T	T	
Chimpanzee	A	C	A			T	T	A	T	G	G	A	C	A	G	G	T	A	A	G	T	A	A	A	A	A	A	C	A	T	A	T	T
Macaque	A	T	A	T	A	C	A	T	T	A	C	G	G	A	C	A	G	G	T	A	A	G	T	A	A	A	A	A	C	A	T		

The Problems are the Same!



Longest Common Subsequence Problem: *Find a longest common subsequence of two strings.*

- **Input:** Two strings.
- **Output:** A longest common subsequence of these strings.

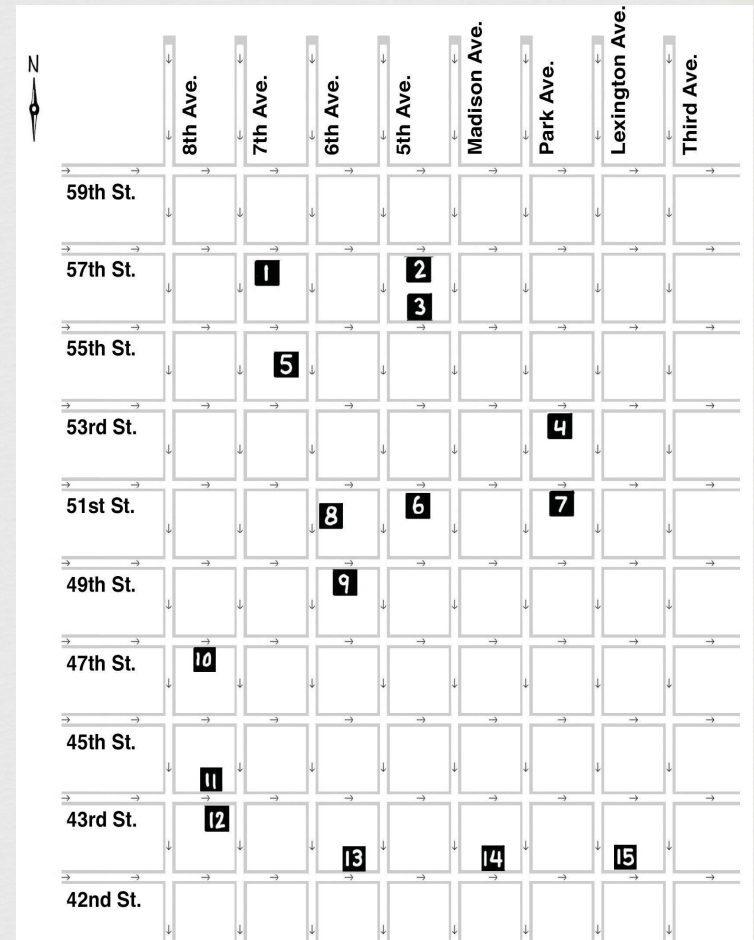
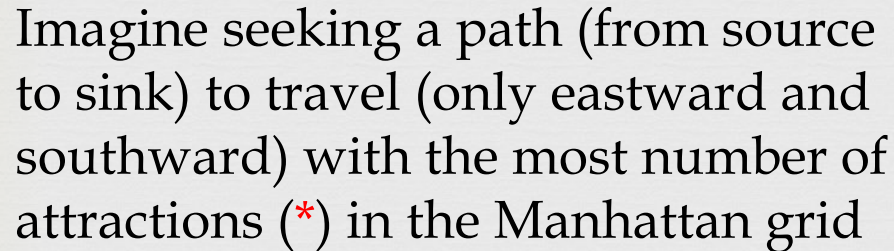
Symbol Matching Problem: *Match as many symbols as possible between two strings.*

- **Input:** Two strings.
- **Output:** The greatest number of matched symbols in any “alignment” of the two strings.

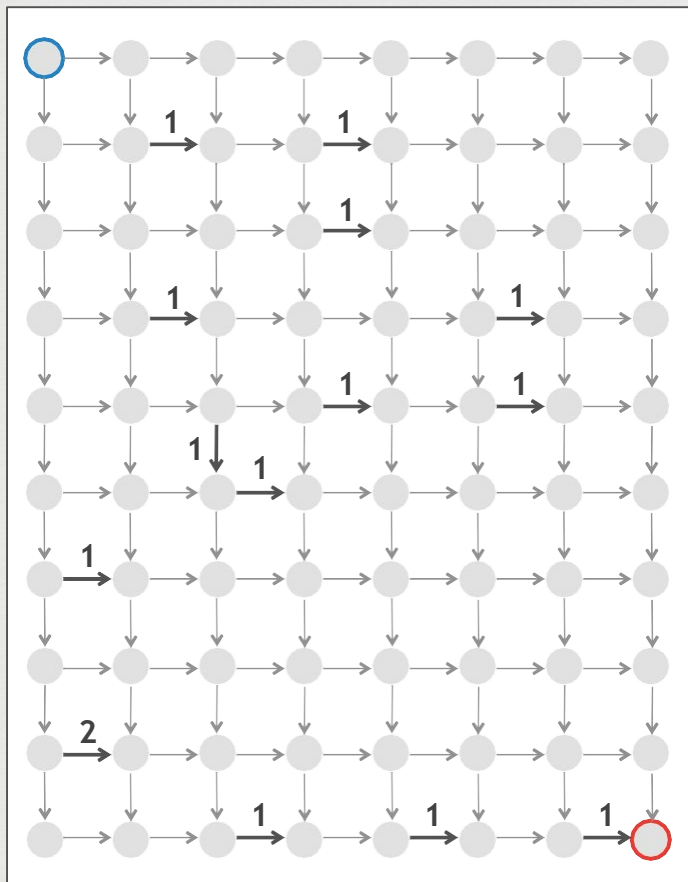
Outline



- Introduction to Sequence Alignment
- **The Manhattan Tourist Problem**
- Sequence Alignment is the Manhattan Tourist Problem in Disguise
- An Introduction to Dynamic Programming: The Manhattan Tourist Problem Revisited
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment



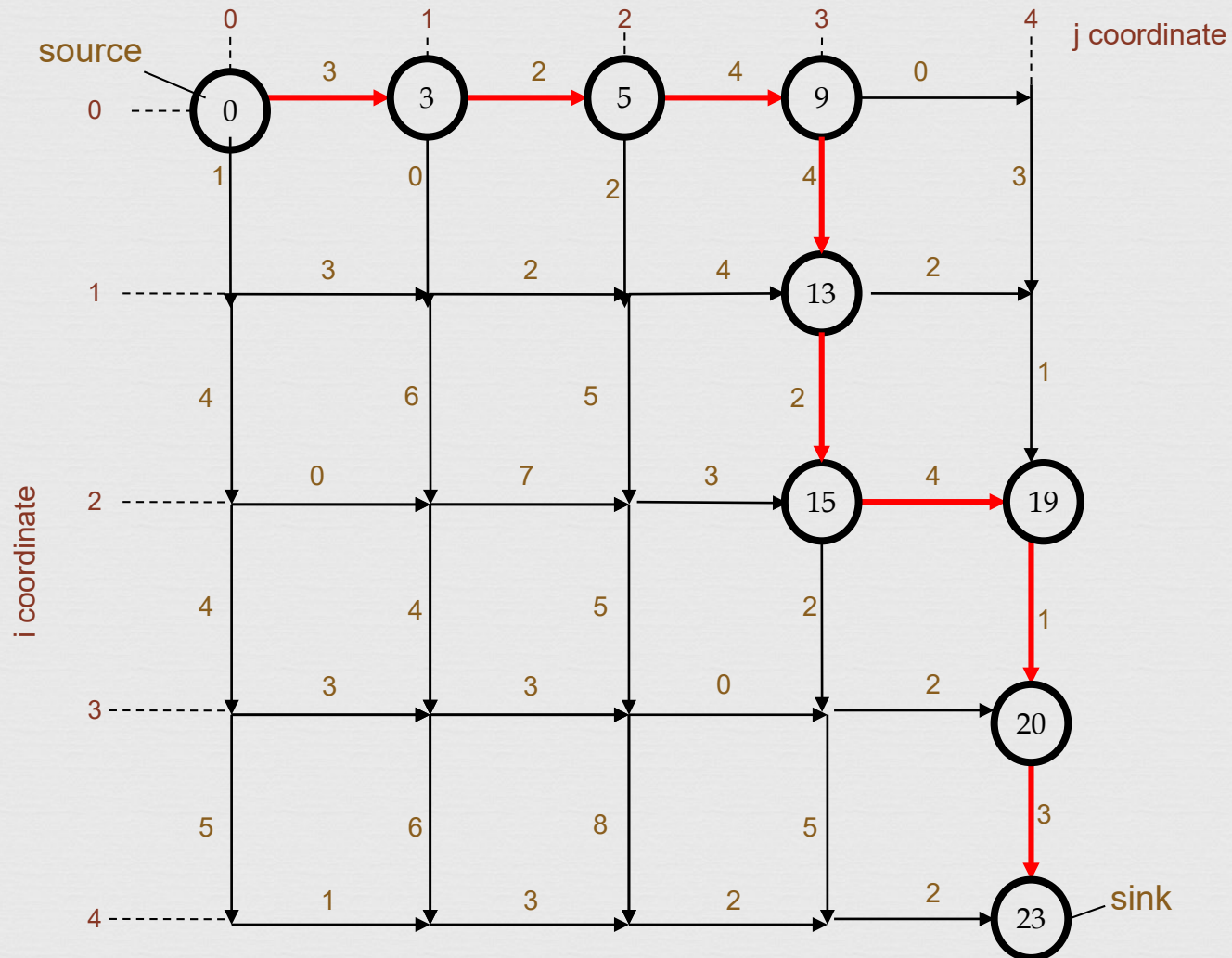
Manhattan Tourist as a Network



Weight of edge:
number of attractions
along the edge.

Goal: Find a **longest**
path from *source* (top
left) to *sink* (bottom
right).

MTP: An Example

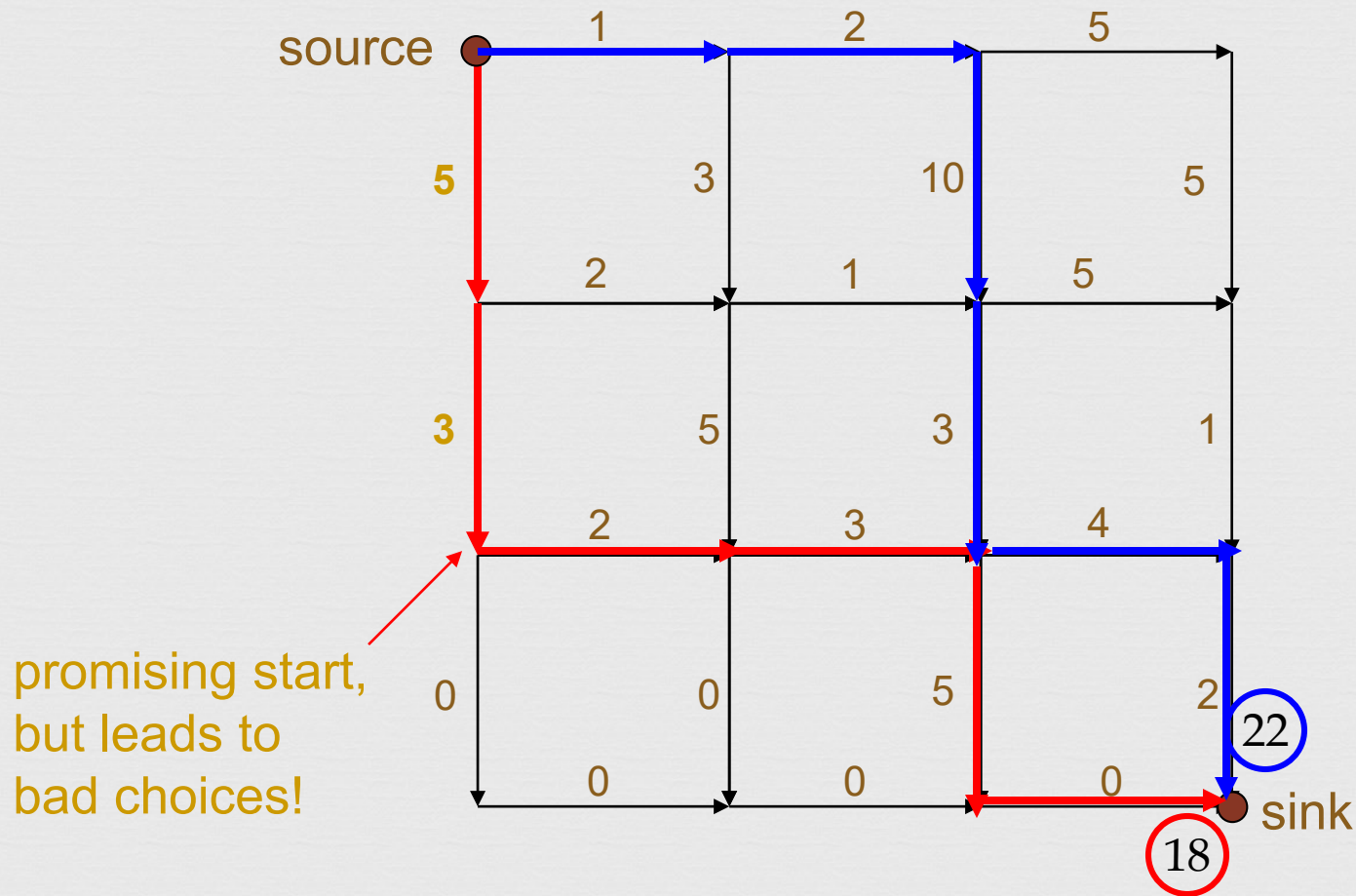


MTP: Greedy Algorithm?



STOP and Think: What's the problem with greedy algorithm for MTP?

MTP: Greedy Algorithm Is Not Optimal



Toward a Computational Problem

Manhattan Tourist Problem: *Find a longest path in a rectangular city.*

- **Input:** A weighted $n \times m$ rectangular grid ($n + 1$ rows and $m + 1$ columns).
- **Output:** A longest path from source $(0, 0)$ to sink (n, m) in the grid.

MTP: Simple Recursive Program



MT(n,m)

if $n=0$ and $m=0$

return 0

if $n>0$

$x \leftarrow \text{MT}(n-1,m)+$

length of the edge from $(n-1,m)$ to (n,m)

if $m>0$

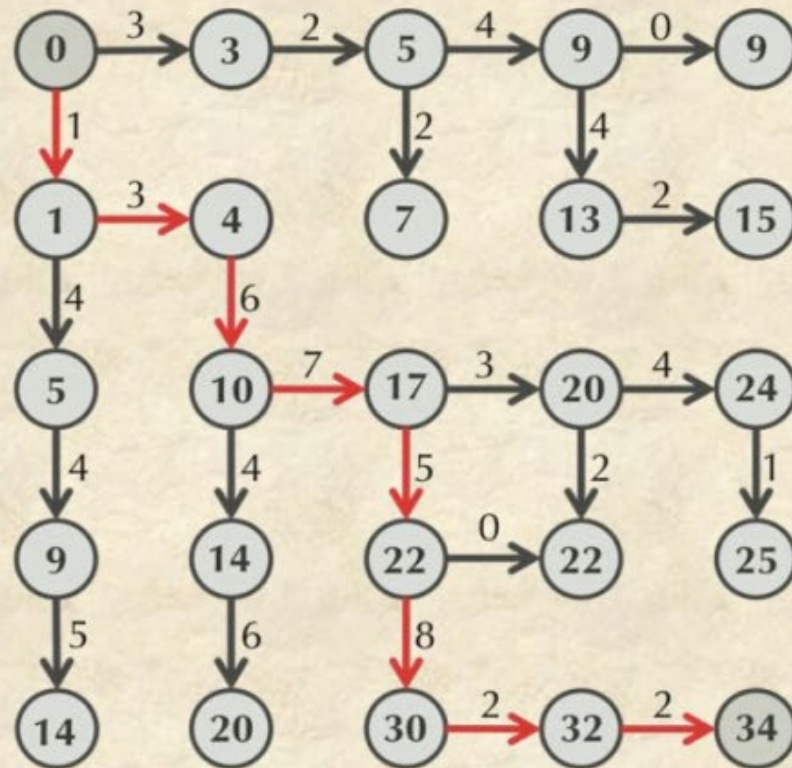
$y \leftarrow \text{MT}(n,m-1)+$

length of the edge from $(n,m-1)$ to (n,m)

return $\max\{x,y\}$

What's **wrong** with this approach?

Reconstructing an Optimal Path



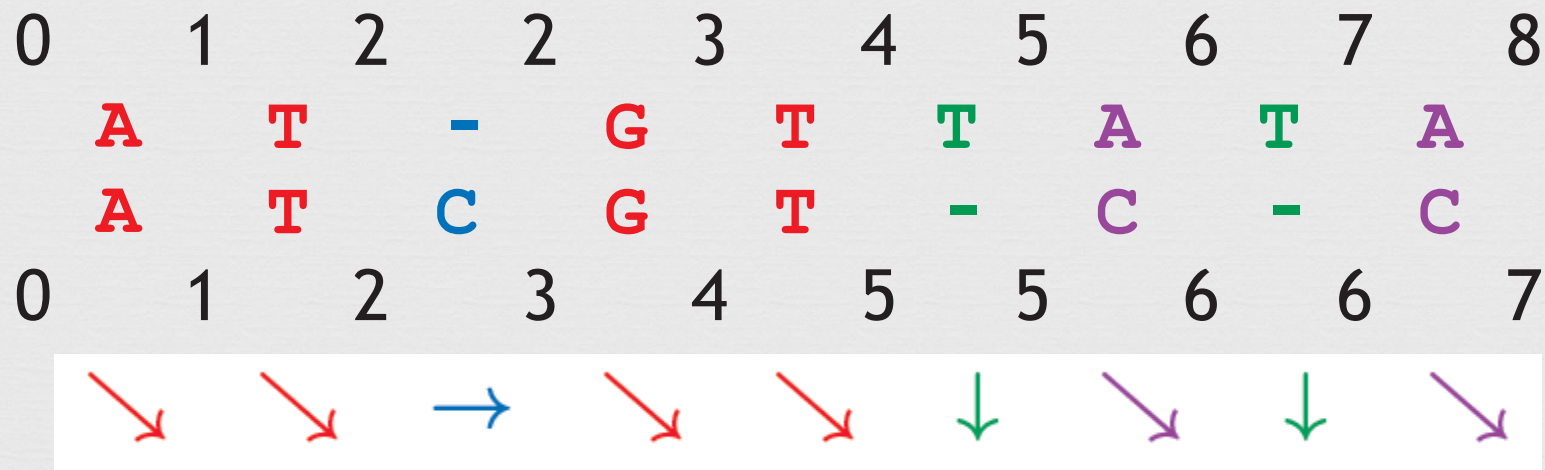
Answer: start at *ending* node and follow edges *backwards* to the beginning node.

Outline



- Introduction to Sequence Alignment
- The Manhattan Tourist Problem
- **Sequence Alignment is the Manhattan Tourist Problem in Disguise**
- An Introduction to Dynamic Programming: The Manhattan Tourist Problem Revisited
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment

Returning to Sequence Alignment ...

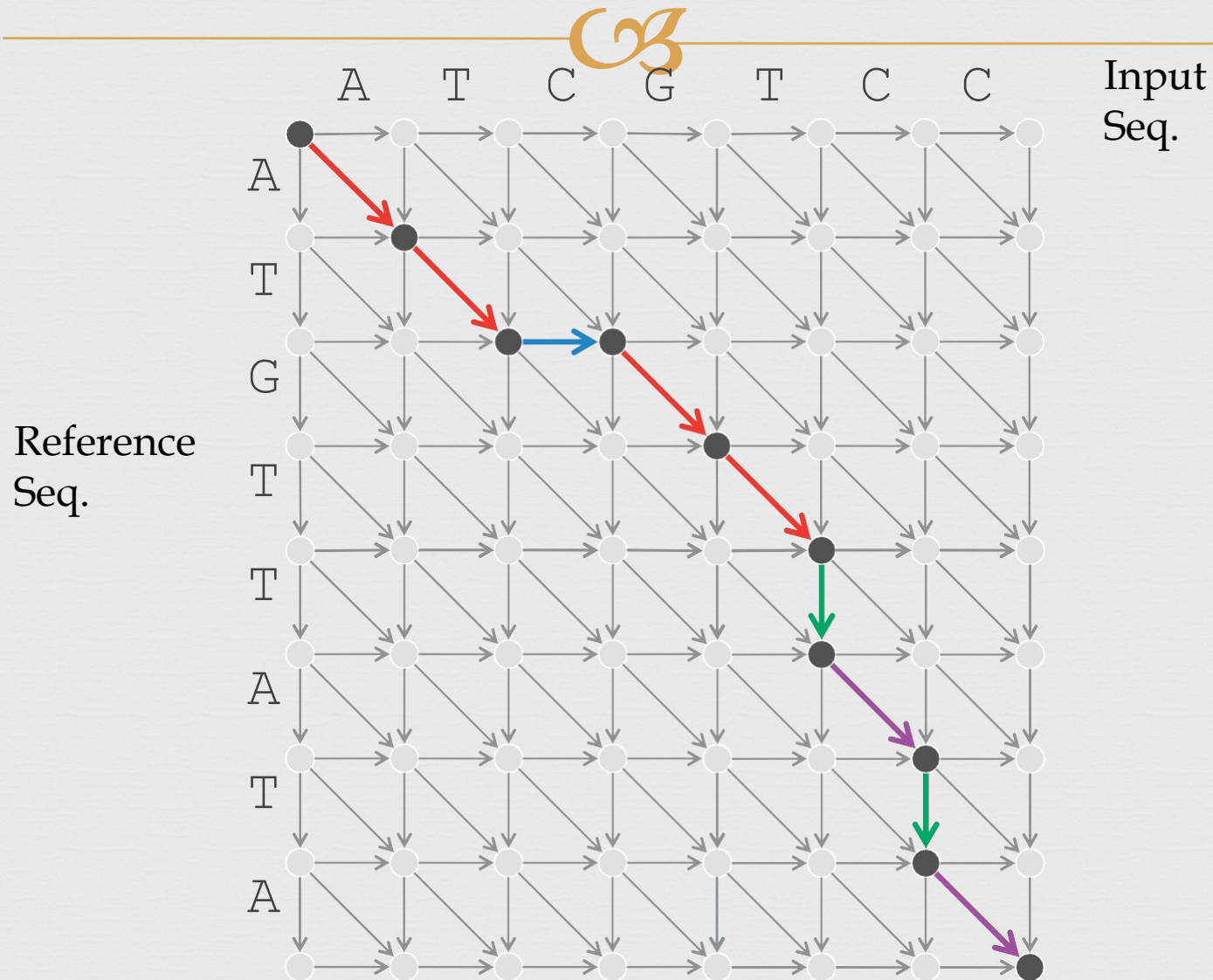


$(0,0) \searrow (1,1) \searrow (2,2) \rightarrow (2,3) \searrow (3,4) \searrow (4,5) \downarrow (5,5) \swarrow (6,6) \downarrow (7,6) \swarrow (8,7)$

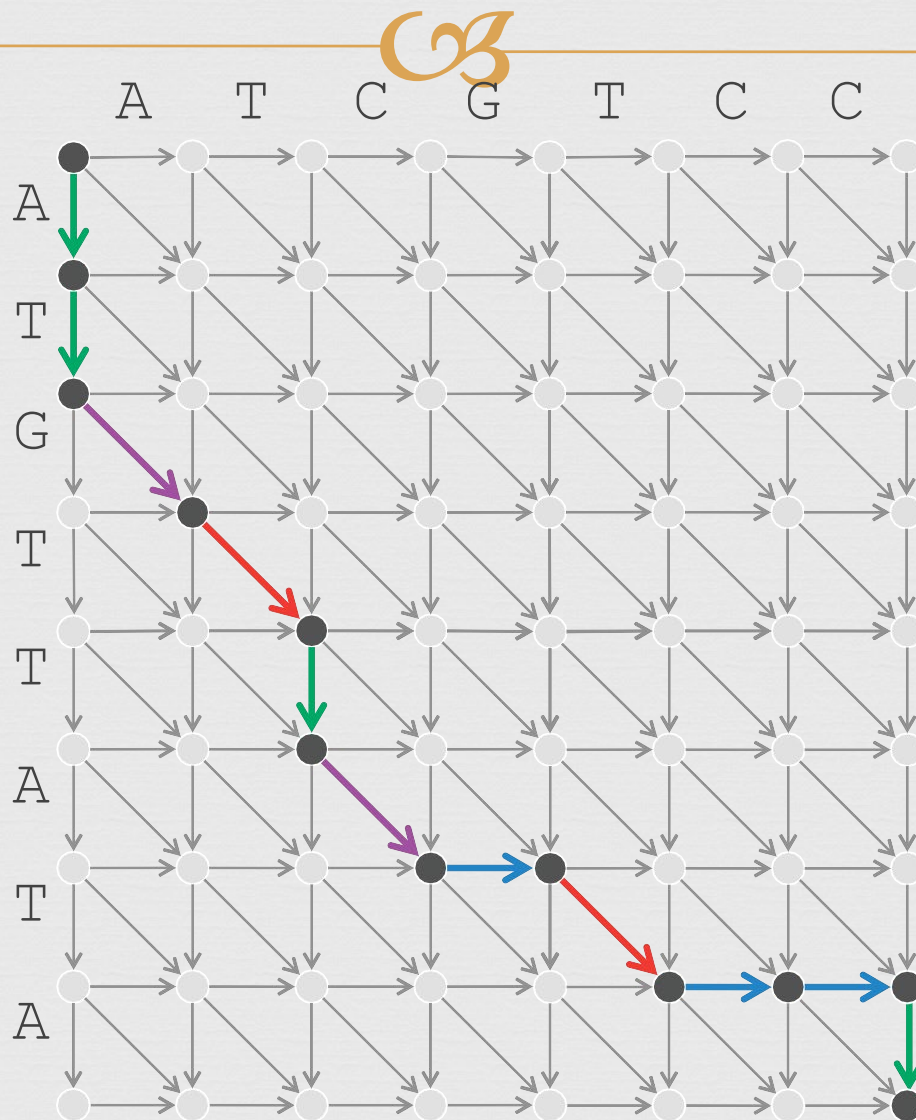
match/mismatch ($\searrow / \swarrow$), **insertion** ($\rightarrow$), or **deletion** ($\downarrow$)

This is a path in a 2-D graph!

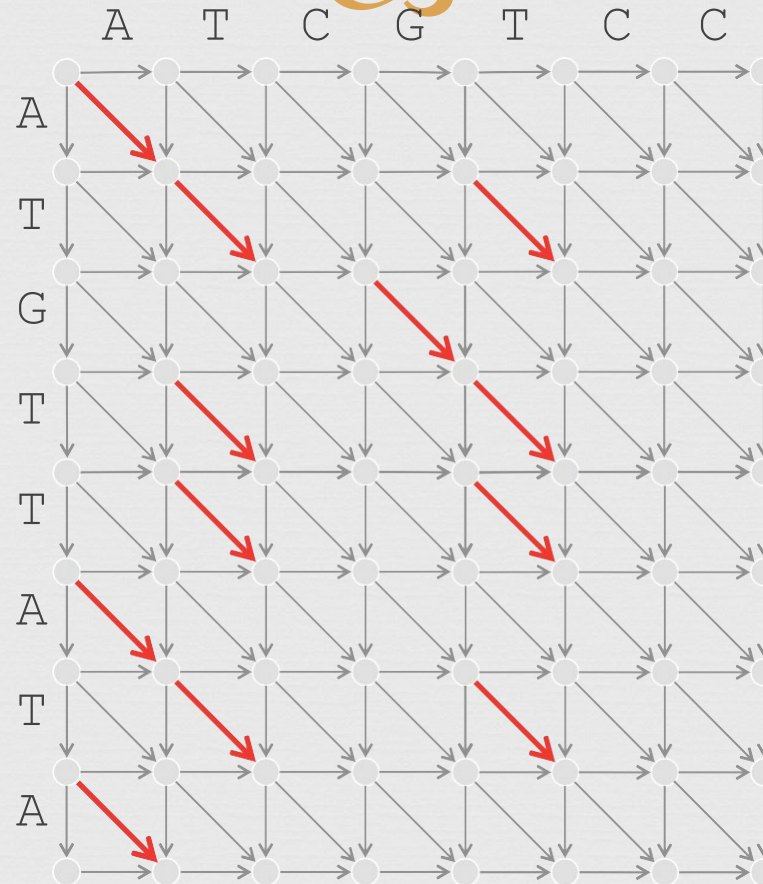
Representing an Alignment as a Path



Reconstruct Alignment from a Path!



Counting Matches Only



STOP and Think: What if we choose to only count the “match” edges in the alignment graph?

The Problems are the Same!



Longest Common Subsequence Problem: *Find a longest common subsequence of two strings.*

- **Input:** Two strings.
- **Output:** A longest common subsequence of these strings.

Symbol Matching Problem: *Match as many symbols as possible between two strings.*

- **Input:** Two strings.
- **Output:** The greatest number of matched symbols in any “alignment” of the two strings.

Algorithm for Longest Path in a DAG?



Still, we haven't discussed how to *find* a longest path in a DAG.

Outline



- Introduction to Sequence Alignment
- The Manhattan Tourist Problem
- Sequence Alignment is the Manhattan Tourist Problem in Disguise
- **An Introduction to Dynamic Programming: The Manhattan Tourist Problem Revisited**
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment

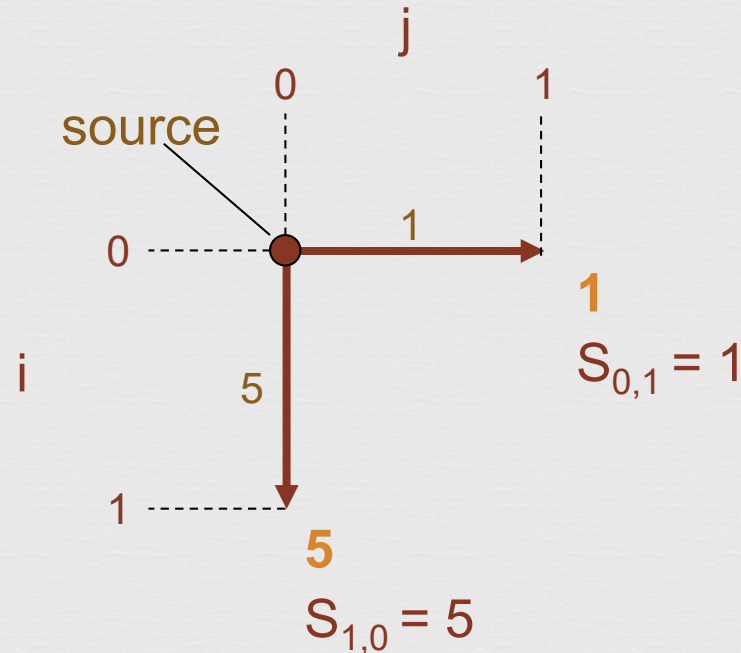
MTP: Dynamic Programming

Computing the score for a point (i,j) by the recurrence relation:

$$s_{i,j} = \max \left\{ \begin{array}{l} s_{i-1,j} + \text{weight of the edge between } (i-1, j) \text{ and } (i, j) \\ s_{i,j-1} + \text{weight of the edge between } (i, j-1) \text{ and } (i, j) \end{array} \right.$$

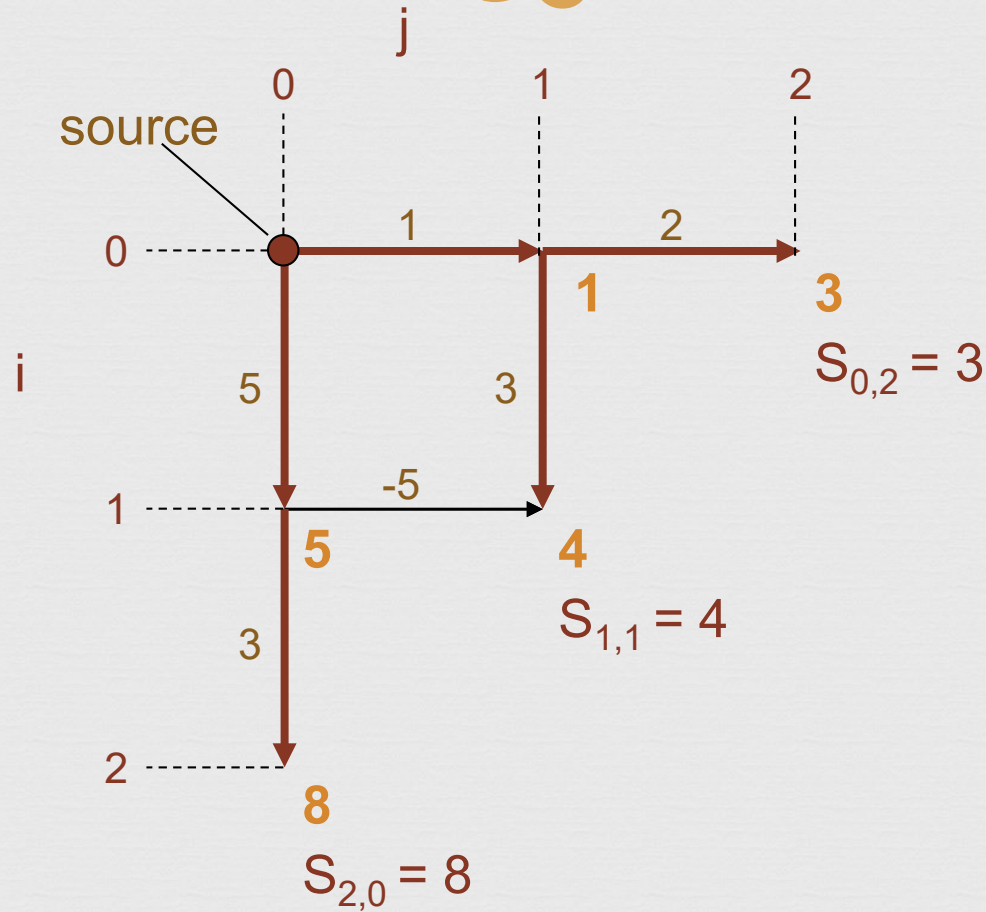
The running time is $\mathbf{n \times m}$ for a $\mathbf{n}$ by $\mathbf{m}$ grid
($\mathbf{n}$ = # of rows, $\mathbf{m}$ = # of columns)

MTP: Dynamic Programming

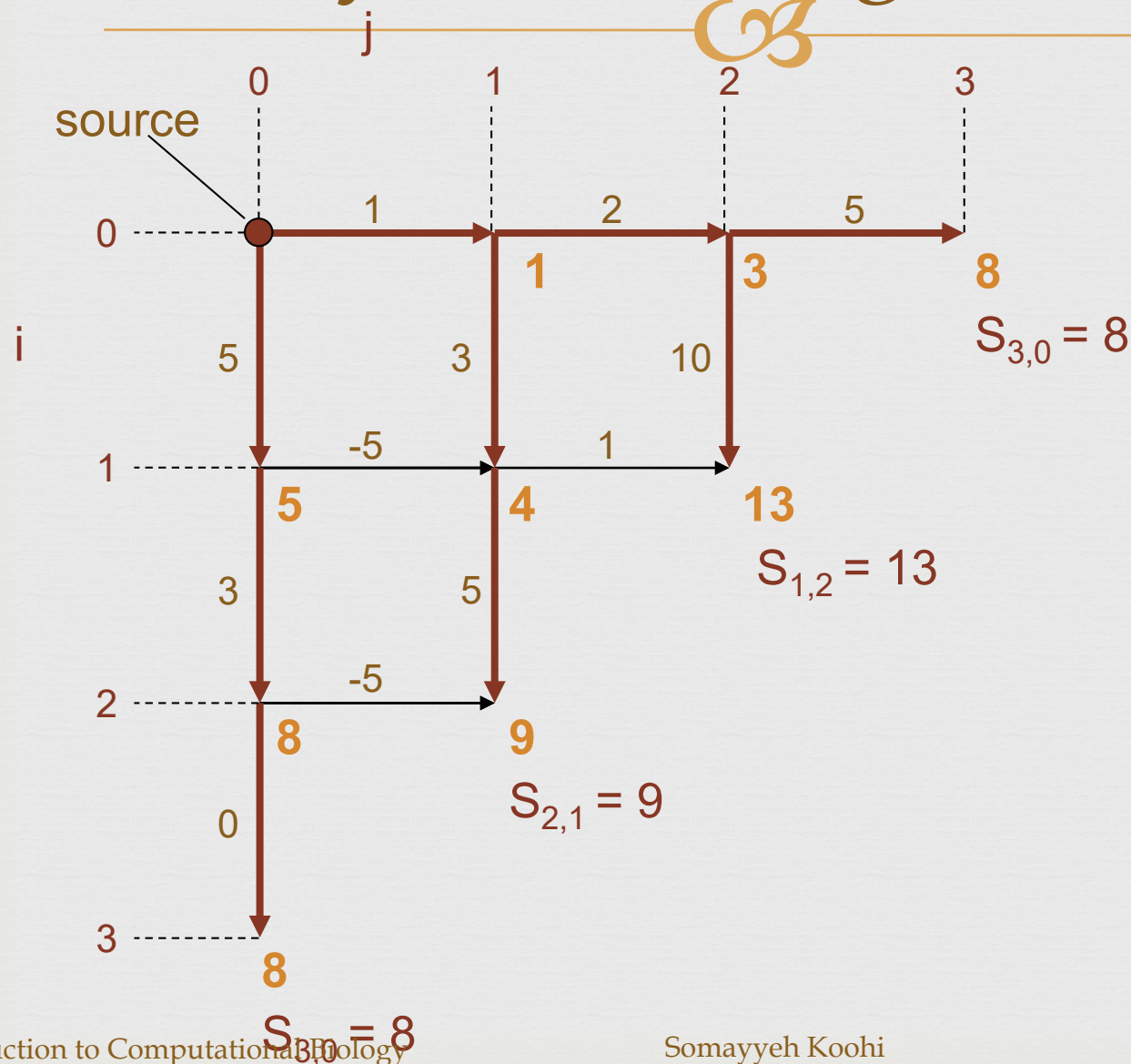


- Calculate optimal path score for each vertex in the graph
- Each vertex's score is the maximum of the prior vertices score plus the weight of the respective edge in between

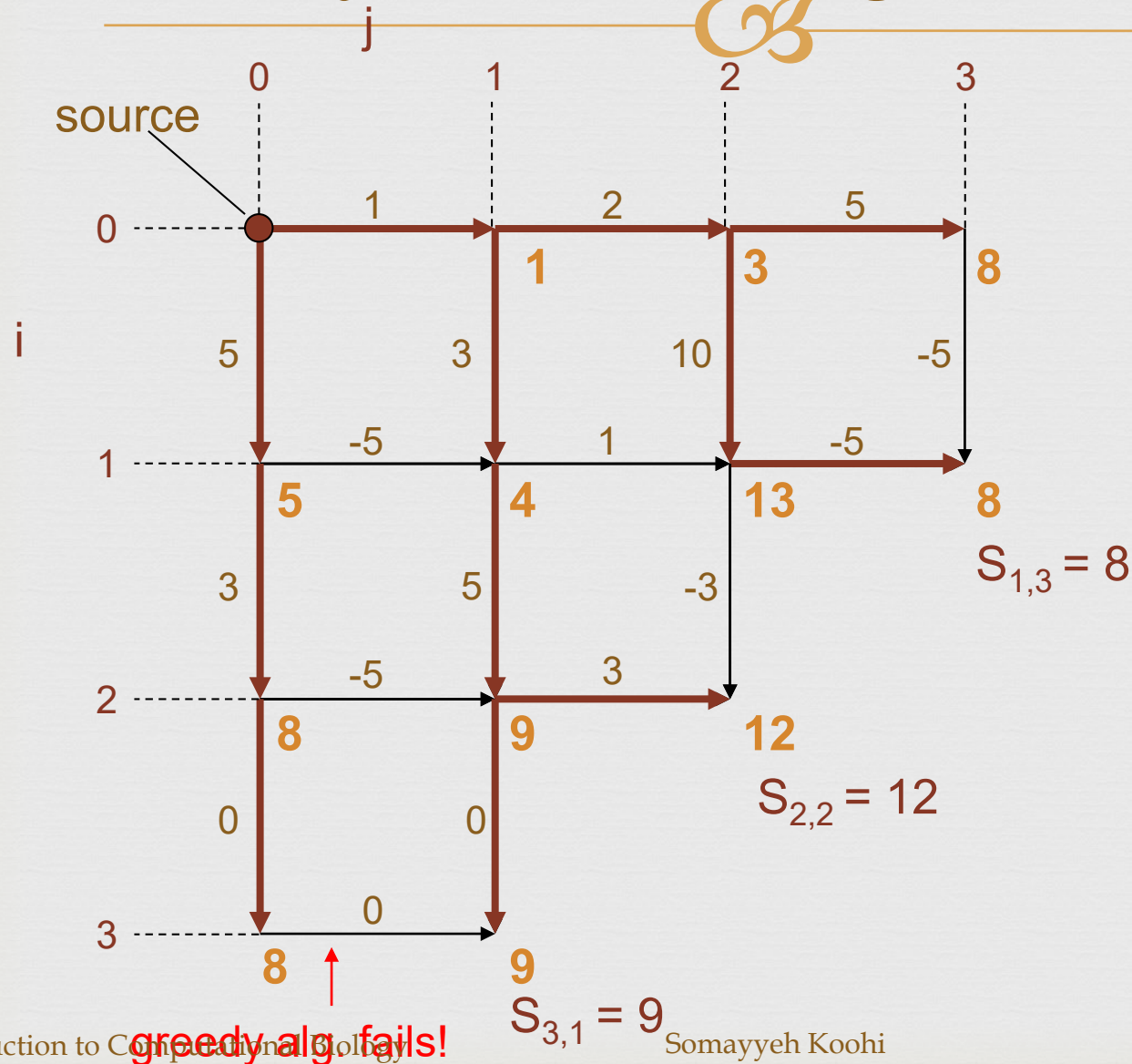
MTP: Dynamic Programming (cont'd)



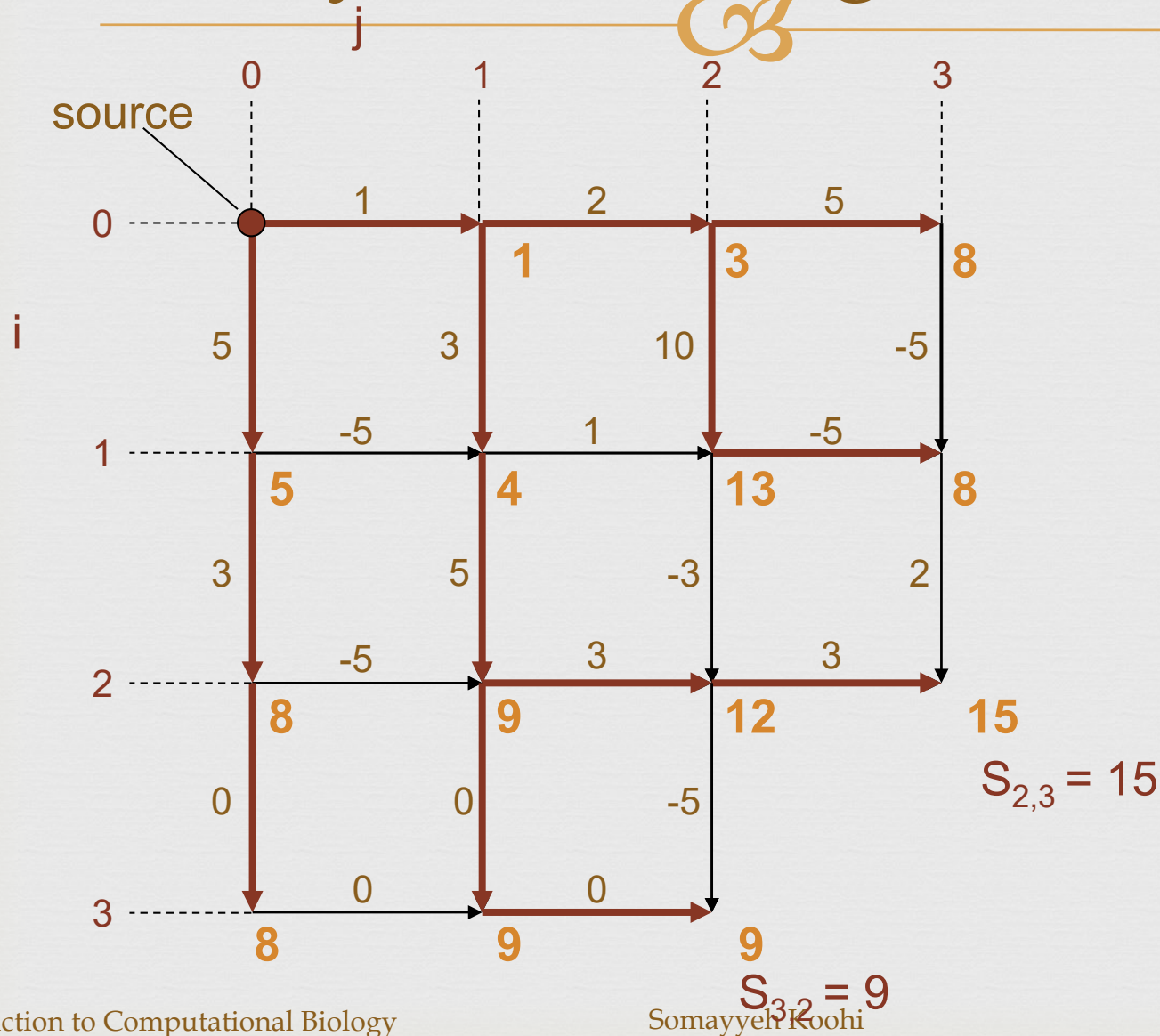
MTP: Dynamic Programming (cont'd)



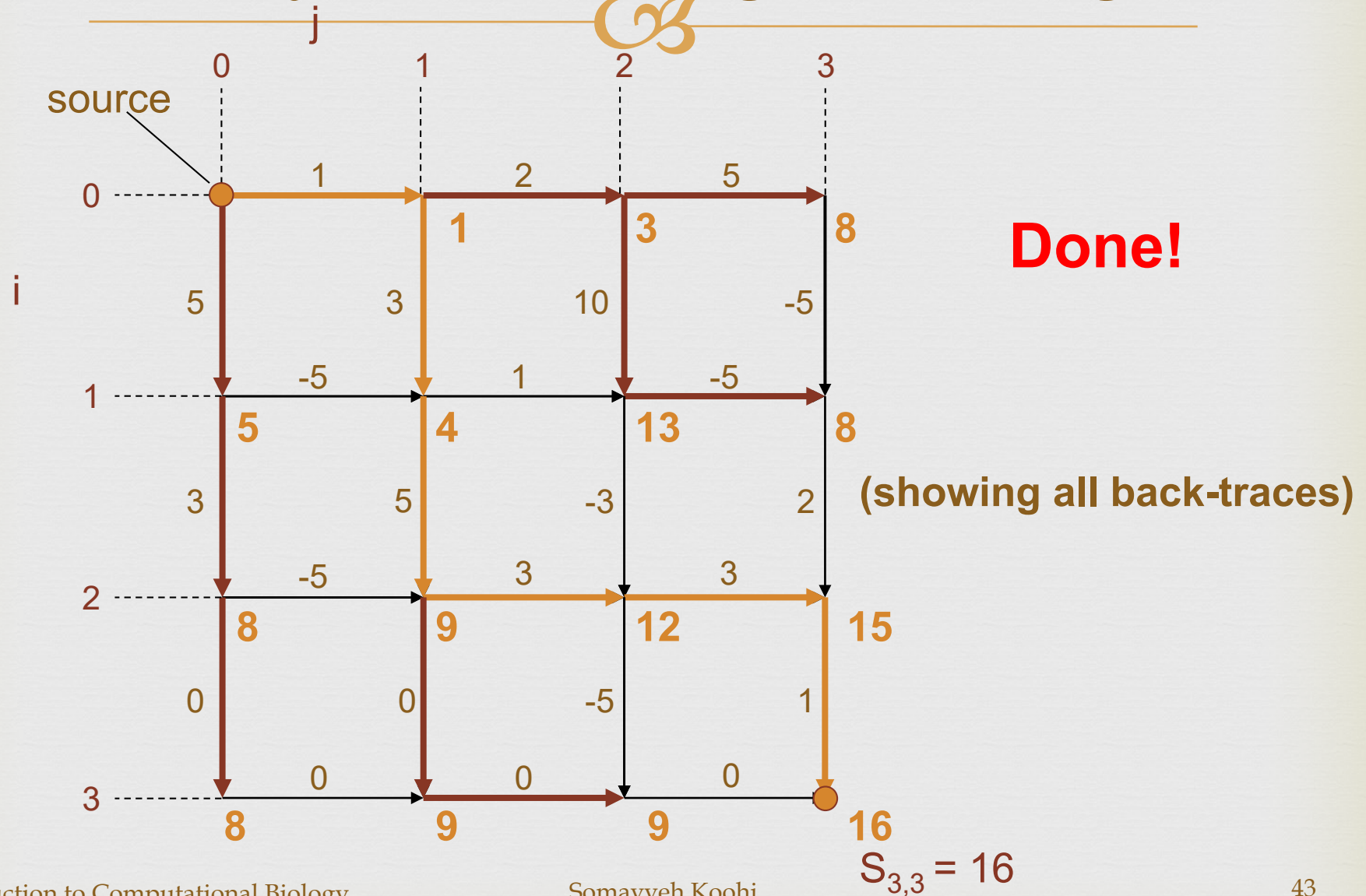
MTP: Dynamic Programming (cont'd)



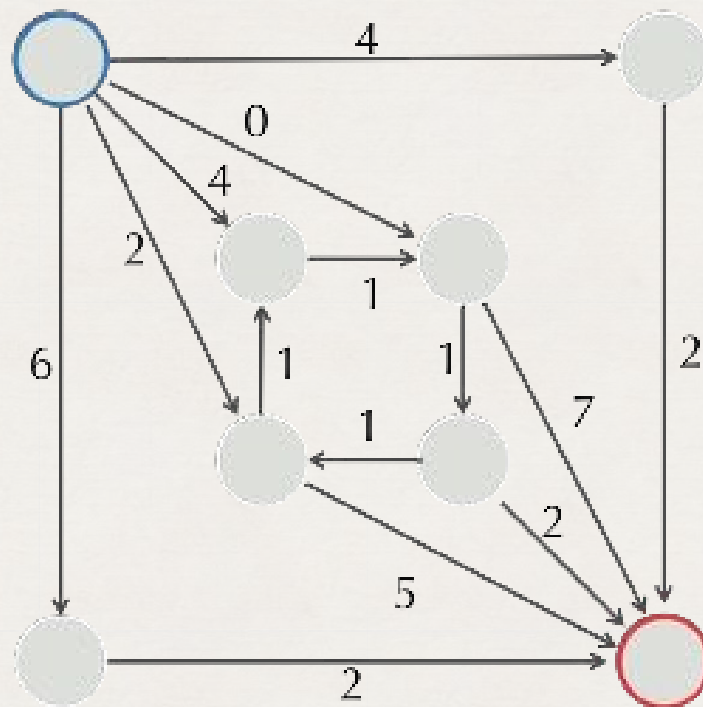
MTP: Dynamic Programming (cont'd)



MTP: Dynamic Programming (cont'd)



Manhattan Is Not A Perfect Grid



STOP and Think: Cycles in graphs cause us major problems, why?!

Generalizing Manhattan Tourist



Directed acyclic graph (DAG): A directed graph that contains *no* cycles.

Longest Path in a DAG Problem : *Find a longest path between two nodes in a **DAG**.*

- **Input:** An edge-weighted DAG with source and sink nodes.
- **Output:** A longest path from source to sink in the DAG.

Generalizing Manhattan Tourist

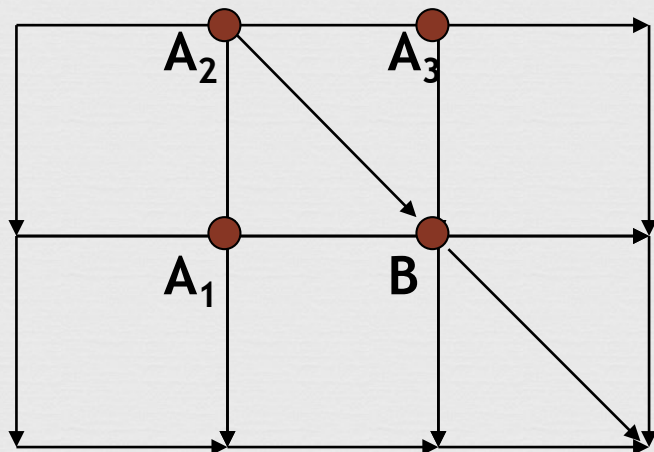


Computing the score for point x is given by the recurrence relation:

$$s_x = \max_{\text{of}} \left\{ s_y + \text{weight of vertex } (y, x) \text{ where } y \in \text{Predecessors}(x) \right.$$

- Predecessors (x) – set of vertices that have edges leading to x
- The running time for a graph $G(V, E)$
(V is the set of all vertices and E is the set of all edges)
is $O(E)$ since each edge is evaluated once

DP in an Arbitrary DAG



What about diagonals?

- The score at point B is given by:

$$s_B = \max \text{ of } \begin{cases} s_{A_1} + \text{weight of the edge } (A_1, B) \\ s_{A_2} + \text{weight of the edge } (A_2, B) \\ s_{A_3} + \text{weight of the edge } (A_3, B) \end{cases}$$

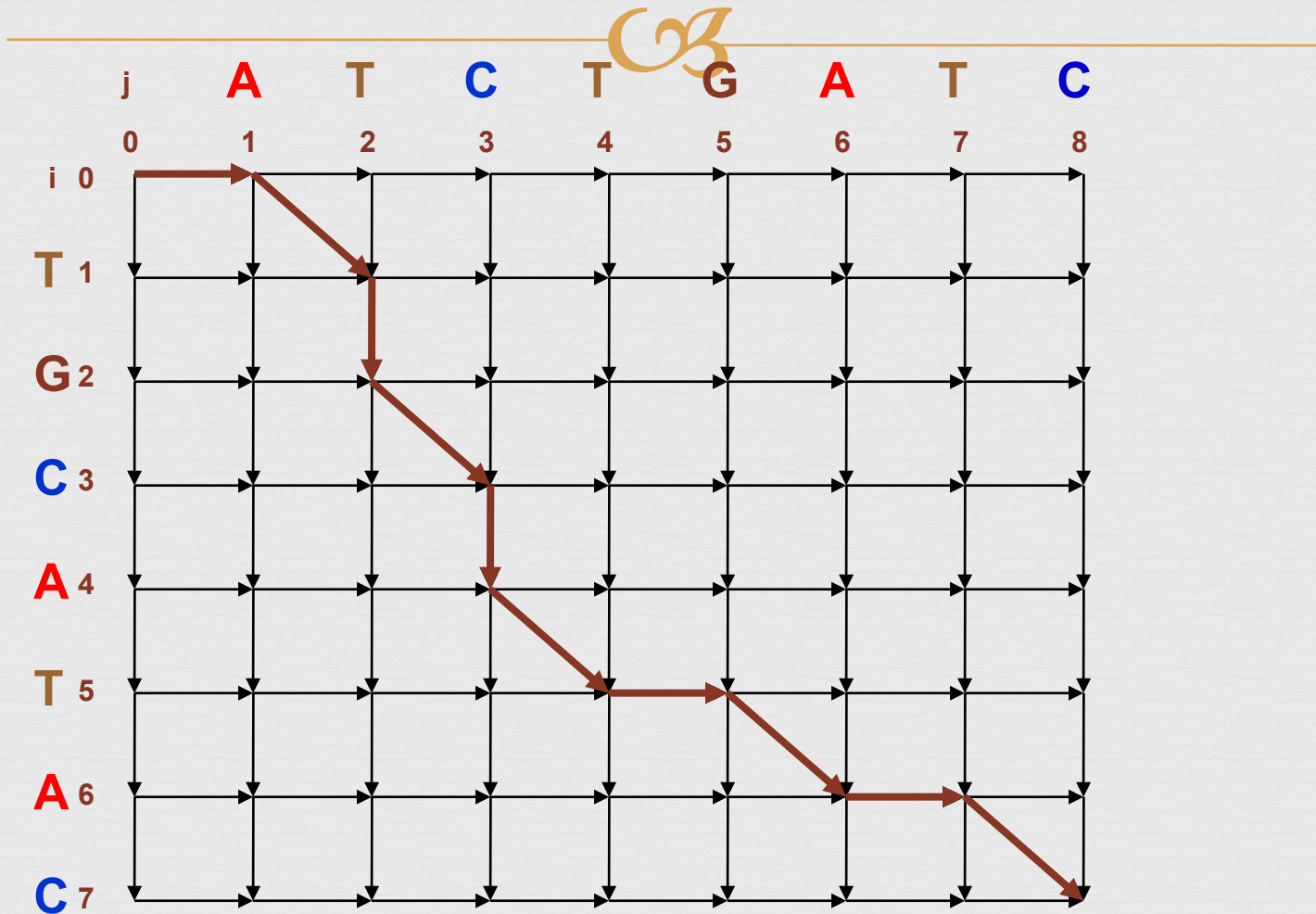
DP in an Arbitrary DAG



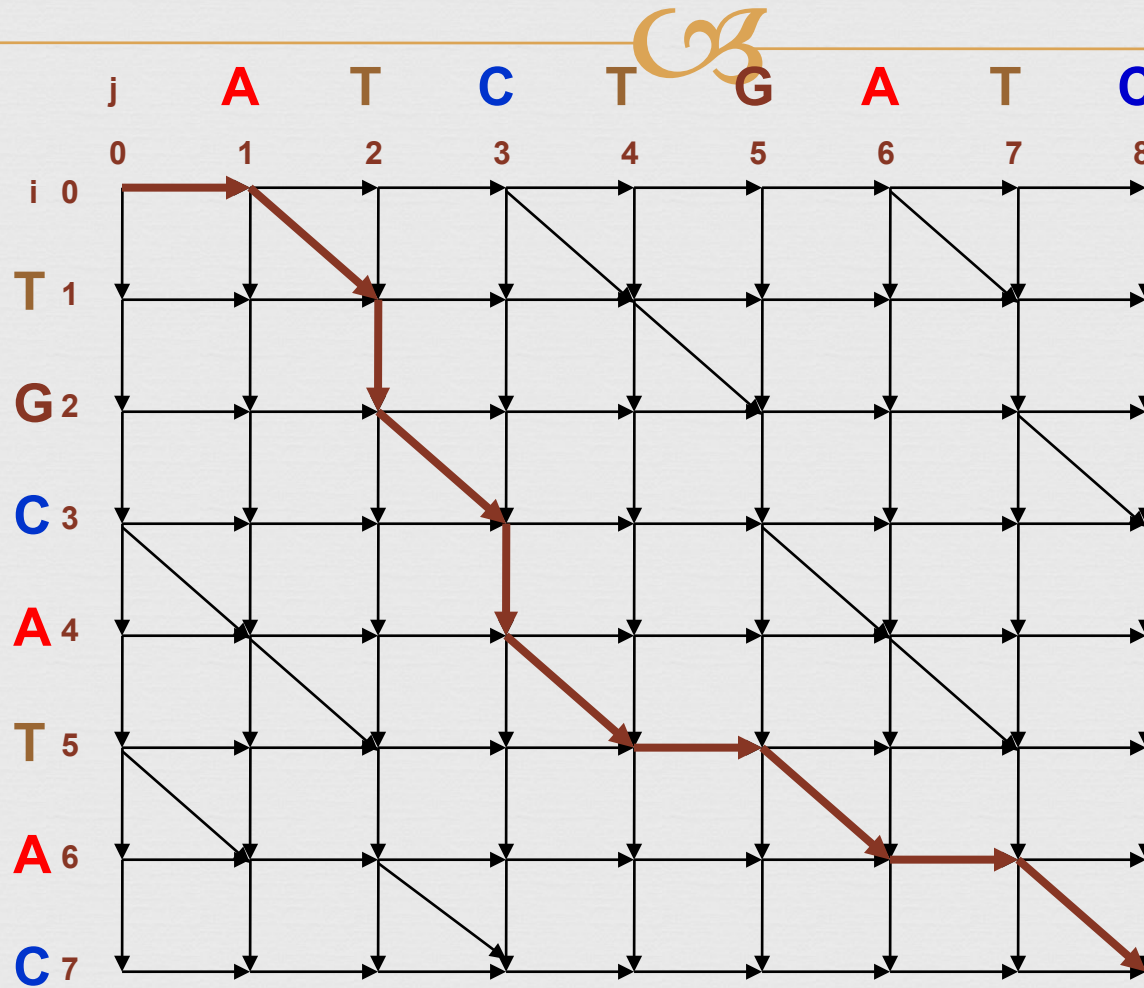
Compute the length of an LCS between sequences v and w

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + \text{weight of edge } \downarrow \text{ between } (i-1, j) \text{ and } (i, j) \\ s_{i,j-1} + \text{weight of edge } \rightarrow \text{ between } (i, j-1) \text{ and } (i, j) \\ s_{i-1,j-1} + \text{weight of edge } \searrow \text{ between } (i-1, j-1) \text{ and } (i, j) \end{cases}$$

LCS Problem as Manhattan Tourist Problem



Edit Graph for LCS Problem



Every path is a common subsequence.

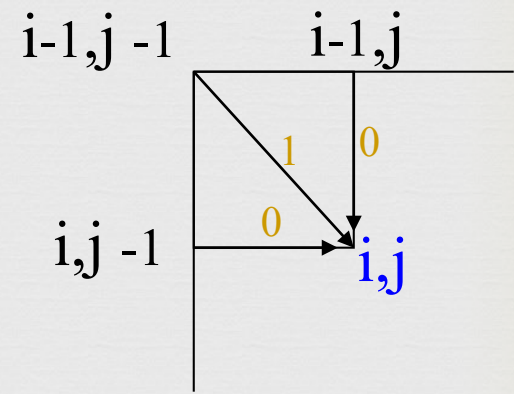
Every diagonal edge adds an extra element to common subsequence

LCS Problem:
Find a path with maximum number of diagonal edges

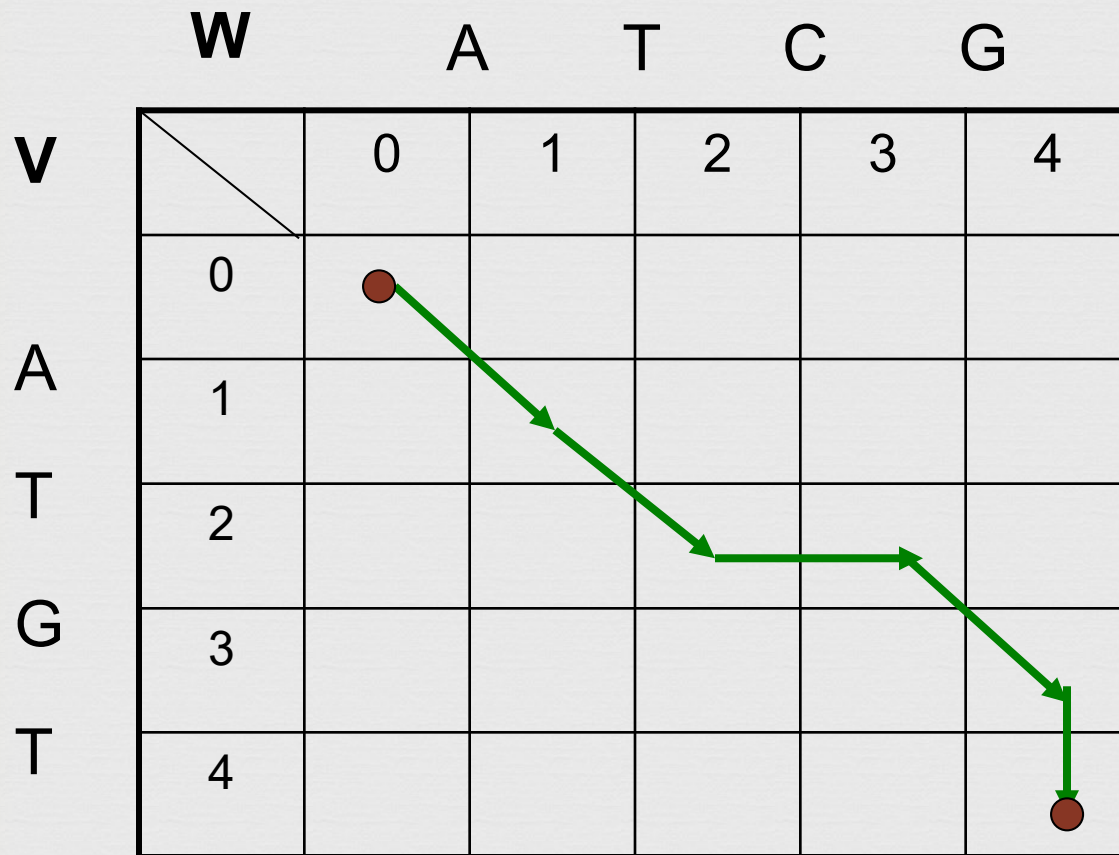
Computing LCS

Let v_i = prefix of v of length i : $v_1 \dots v_i$
 and w_j = prefix of w of length j : $w_1 \dots w_j$
 The length of $\text{LCS}(v_i, w_j)$ is computed by:

$$s_{i,j} = \max \begin{cases} s_{i-1,j} \\ s_{i,j-1} \\ s_{i-1,j-1} + 1 \text{ if } v_i = w_j \end{cases}$$

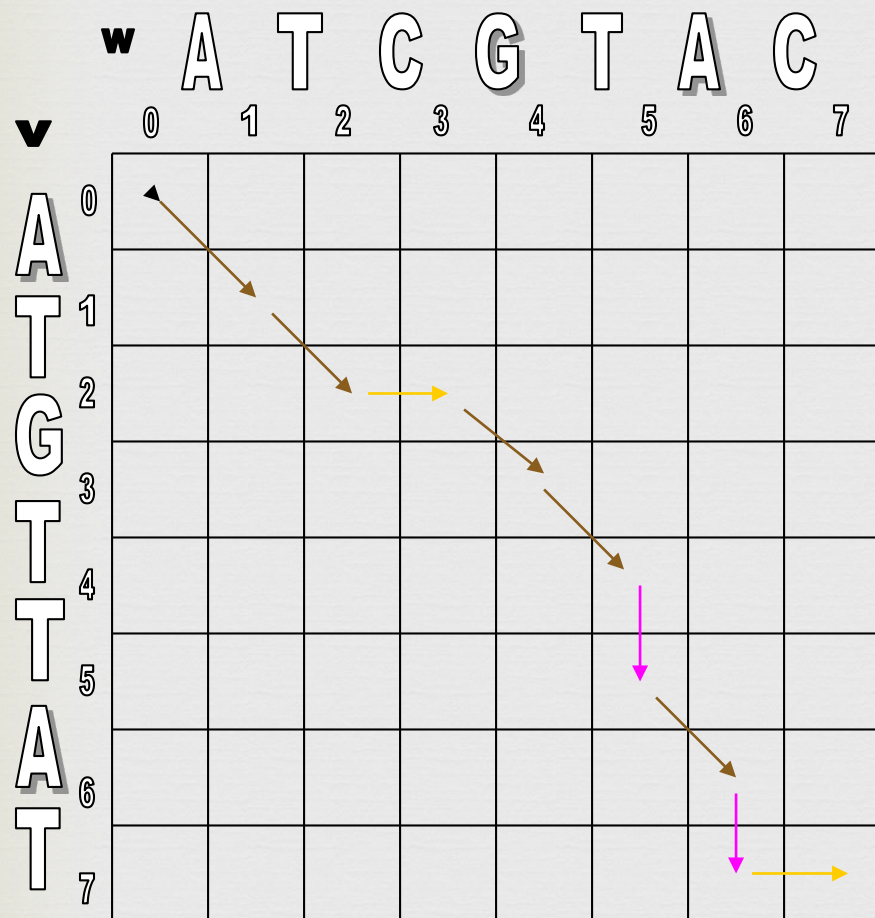


Every Path in the Grid Corresponds to an Alignment



$\swarrow \searrow \rightarrow \swarrow \downarrow$
 0 1 2 2 3 4
 V = AT - GT
 | | |
 W = ATCG -
 0 1 2 3 4 4

Alignment as a Path in the Graph



012**2**34**5**6**7**7

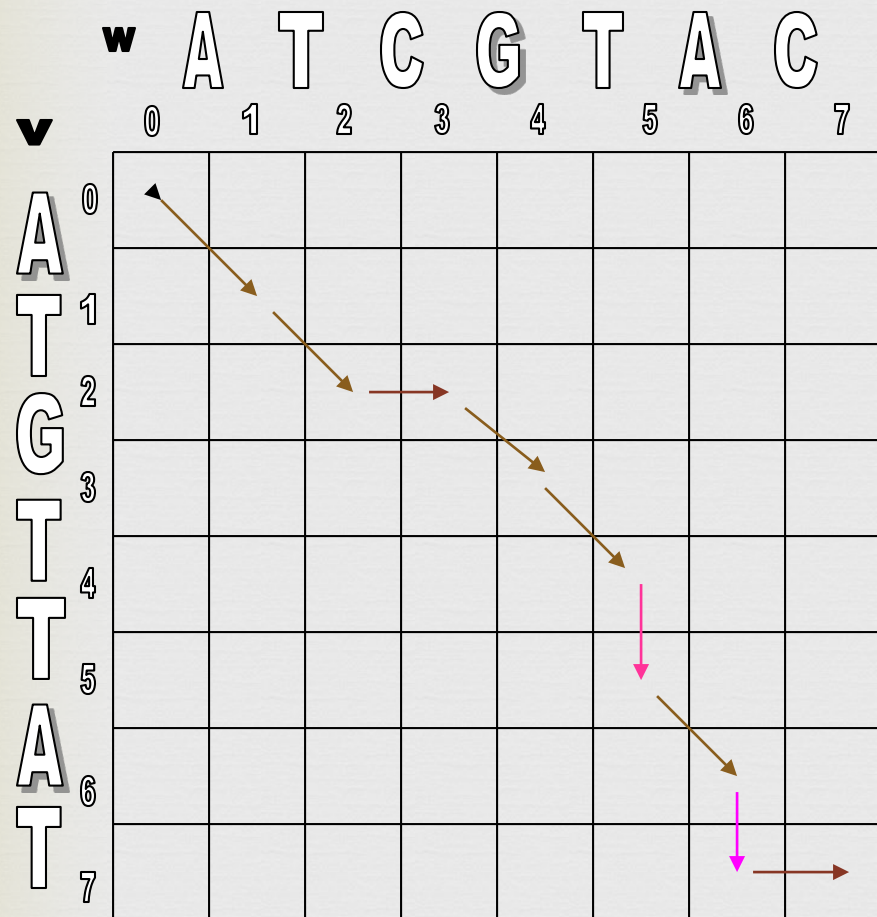
V= AT_**G**TT**A**T_

W= AT**C**GT_**A**_C

012**3**45**5**6**6**7

(0,0) , (1,1) , (2,2), **(2,3)**,
(3,4), (4,5), **(5,5)**, (6,6),
(7,6), **(7,7)**

Alignment as a Path in the Graph

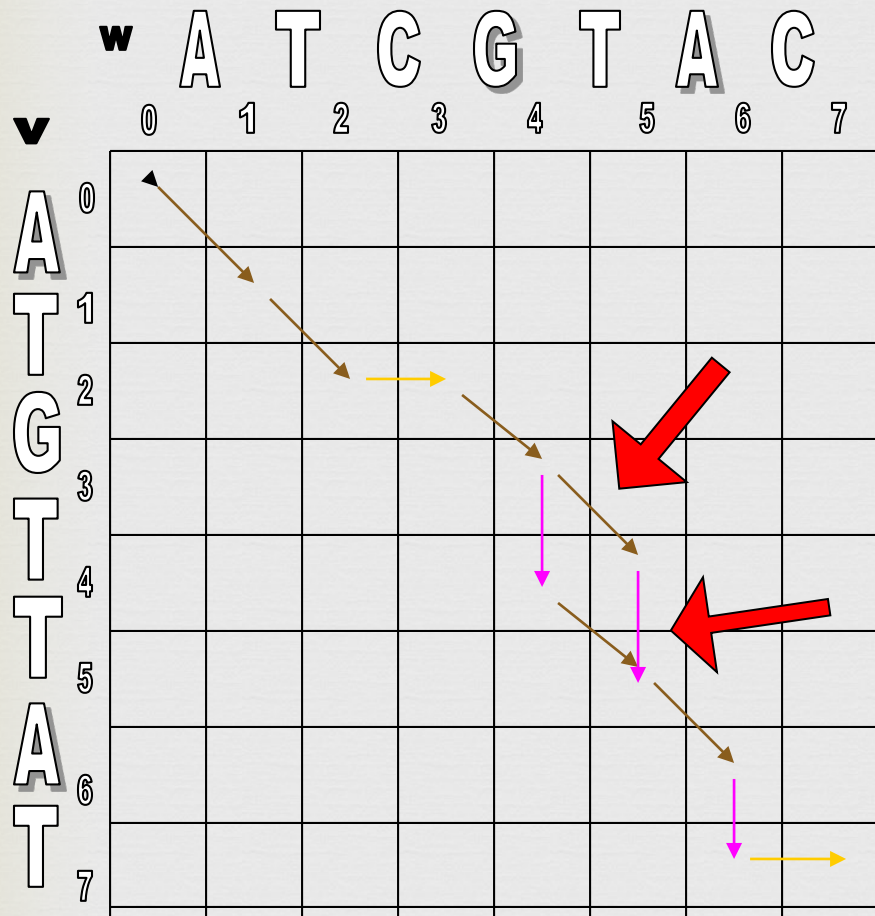


↓ and → represent indels in **v** and **w** with score 0.

↘ represent matches with score 1.

- The score of the alignment path is 5.

Alignment as a Path in the Graph



Old Alignment

0122345677

V= AT_GTTAT_

W= ATCGT_A_C

0123455667

New Alignment

0122345677

V= AT_GTTAT_

W= ATCG_TA_C

0123445667

Alignment: Dynamic Programming



$$s_{i,j} = \max \left\{ \begin{array}{l} s_{i-1,j-1} + 1 \text{ if } v_i = w_j \\ s_{i-1,j} \\ s_{i,j-1} \end{array} \right.$$

Diagram illustrating the dynamic programming recurrence for sequence alignment. The recurrence is shown as a maximum of three cases:

- $s_{i-1,j-1} + 1$ if $v_i = w_j$ (diagonal arrow)
- $s_{i-1,j}$ (vertical arrow)
- $s_{i,j-1}$ (horizontal arrow)

Dynamic Programming Example



		w							
		A	T	C	G	T	A	C	
		0	1	2	3	4	5	6	7
v	A	0	0	0	0	0	0	0	0
	T	0							
	G	0							
	T	0							
	T	0							
	T	0							
	A	0							
	T	0							

Initialize 1st row and 1st column to be all zeroes.

Or, to be more precise, initialize 0th row and 0th column to be all zeroes.

Dynamic Programming Example



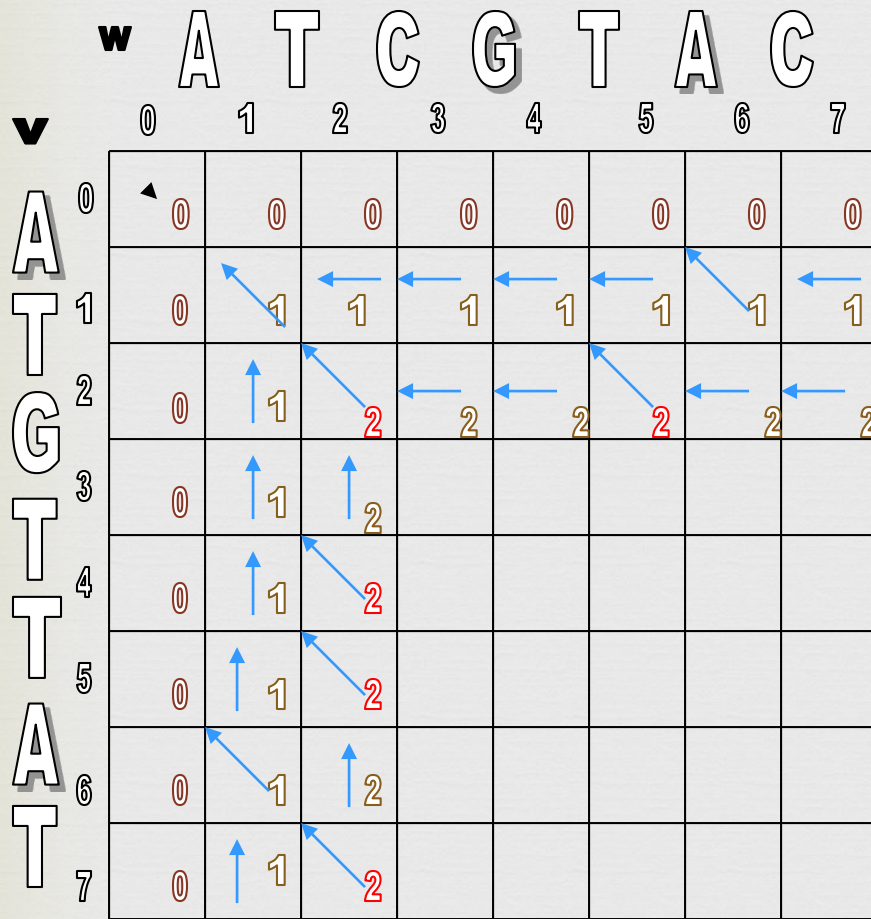
		w							
		A	T	C	G	T	A	C	
		0	1	2	3	4	5	6	7
v	A	0	0	0	0	0	0	0	0
	T	0	1	1	1	1	1	1	1
	G	0	1						
	T	0	1						
	T	0	1						
	T	0	1						
	A	0	1						
	T	0	1						

$$S_{i,j} = \max \begin{cases} S_{i-1,j-1} & \leftarrow \text{value from NW} + 1, \text{ if } v_i = w_j \\ S_{i-1,j} & \leftarrow \text{value from North (top)} \\ S_{i,j-1} & \leftarrow \text{value from West (left)} \end{cases}$$

Arrows show where the score originated from.



Backtracking Example



Find a match in row and column 2.

$i=2, j=2,5$ is a match (T).

$j=2, i=2,4,5,7$ is a match (T).

Since $v_i = w_j$, $s_{i,j} = s_{i-1,j-1} + 1$

$$s_{2,2} = [s_{1,1} = 1] + 1$$

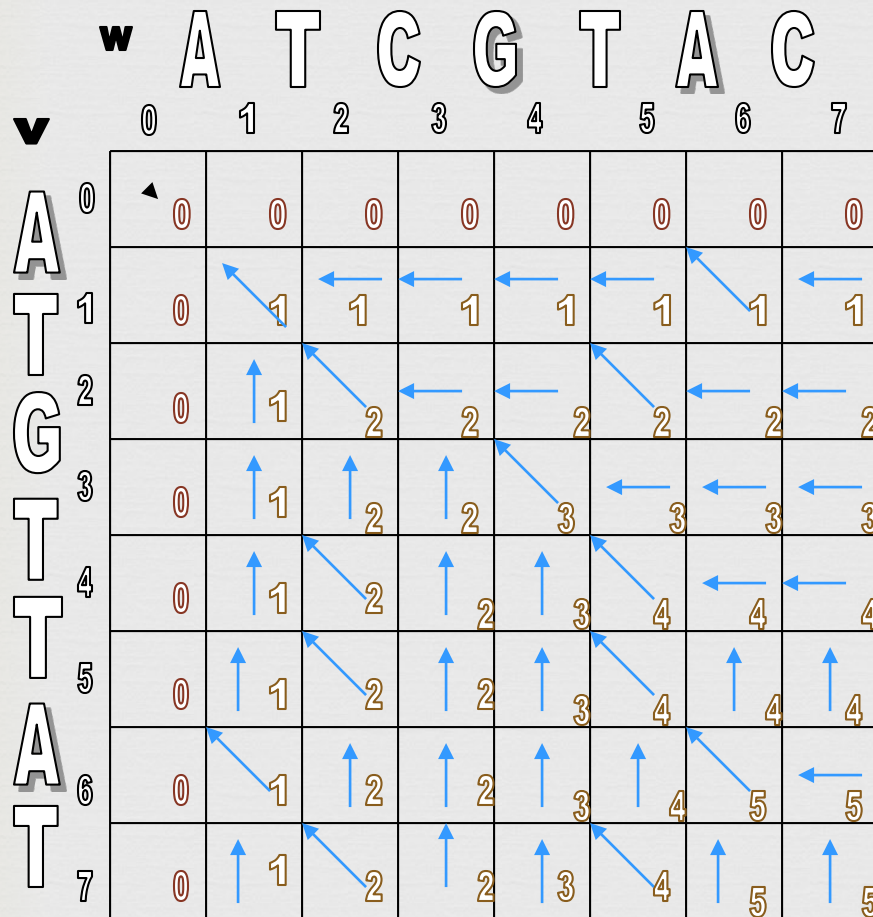
$$s_{2,5} = [s_{1,4} = 1] + 1$$

$$s_{4,2} = [s_{3,1} = 1] + 1$$

$$s_{5,2} = [s_{4,1} = 1] + 1$$

$$s_{7,2} = [s_{6,1} = 1] + 1$$

Backtracking Example



Continuing with the dynamic programming algorithm gives this result.

LCS Algorithm

```

1. LCS(v,w)
2.   for  $i \leftarrow 1$  to  $n$ 
3.      $s_{i,0} \leftarrow 0$ 
4.   for  $j \leftarrow 1$  to  $m$ 
5.      $s_{0,j} \leftarrow 0$ 
6.   for  $i \leftarrow 1$  to  $n$ 
7.     for  $j \leftarrow 1$  to  $m$ 
8.        $s_{i,j} \leftarrow \max \begin{cases} s_{i-1,j} \\ s_{i,j-1} \\ s_{i-1,j-1} + 1, \text{ if } v_i = w_j \end{cases}$ 
9.        $b_{i,j} \leftarrow \begin{cases} \text{"} \uparrow \text{"} & \text{if } s_{i,j} = s_{i-1,j} \\ \text{"} \leftarrow \text{"} & \text{if } s_{i,j} = s_{i,j-1} \\ \text{"} \nwarrow \text{"} & \text{if } s_{i,j} = s_{i-1,j-1} + 1 \end{cases}$ 
10.
11.
∞
∞
∞   return  $(s_{n,m}, b)$ 

```

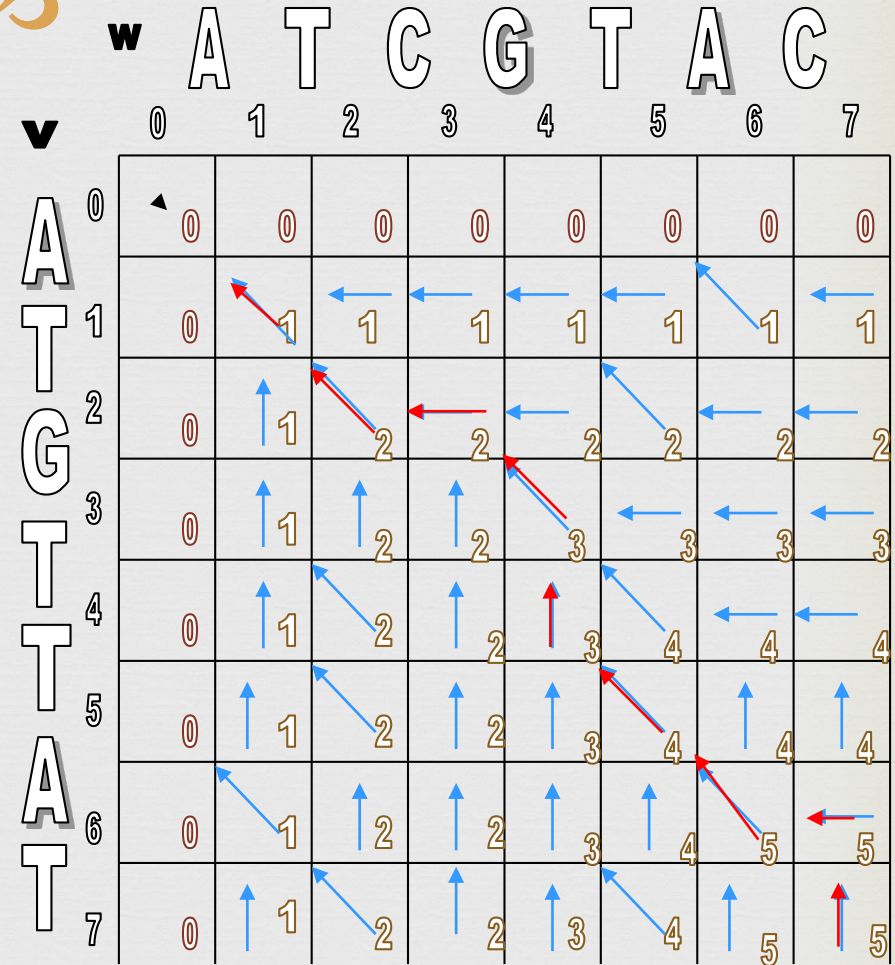
Diagram illustrating the LCS algorithm with arrows indicating the flow of computation and the conditions for updating the DP table:

- A pink arrow points down to $s_{i-1,j}$.
- A yellow arrow points right to $s_{i,j-1}$.
- A brown arrow points down-right to $s_{i-1,j-1} + 1$.
- Upward, leftward, and diagonal arrows point from the corresponding b values to the conditions in the $b_{i,j}$ assignment.

Now What?



- ✧ LCS(v,w) created the alignment grid
- ✧ Now we need a way to read the best alignment of v and w
- ✧ Follow the arrows backwards from sink



Printing LCS: Backtracking



1. **PrintLCS(b,v,i,j)**
2. **if** $i = 0$ or $j = 0$
3. **return**
4. **if** $b_{i,j} = \text{"} \swarrow \text{"}$
5. **PrintLCS(b,v,i-1,j-1)**
6. **print** v_i
7. **else**
8. **if** $b_{i,j} = \text{"} \uparrow \text{"}$
9. **PrintLCS(b,v,i-1,j)**
10. **else**
11. **PrintLCS(b,v,i,j-1)**

LCS Runtime



✧ It takes $O(nm)$ time to fill in the $n \times m$ dynamic programming matrix.

✧ Why $O(nm)$?

Outline



- Introduction to Sequence Alignment
- The Manhattan Tourist Problem
- Sequence Alignment is the Manhattan Tourist Problem in Disguise
- An Introduction to Dynamic Programming: The Manhattan Tourist Problem Revisited
- **From Global to Local Alignment**
- Penalizing Insertions and Deletions in Sequence Alignment

Optimal alignment

- ✧ An alignment can be specified by the traceback matrix.
- ✧ Needleman-Wunsch algorithm for global alignment
 - ✧ First proposed in 1970
 - ✧ Widely used in genomics/bioinformatics
 - ✧ Dynamic programming algorithm
- ✧ Smith-Waterman algorithm for local alignment
 - ✧ Useful in motif detection, database search, short read mapping

Local vs. Global Alignment



- ✧ The Global Alignment Problem tries to find the longest path between vertices $(0,0)$ and (n,m) in the edit graph.
- ✧ The Local Alignment Problem tries to find the longest path among paths between **arbitrary vertices** (i,j) and (i', j') in the edit graph.
 - For example, **positions 20-40** of *sequence A* might be aligned with **positions 50-70** of *sequence B*

Global

G	T	A	G	C	T	T	A	A	G	G	T	T	A
-	T	A	G	-	-	-	A	-	-	-	T	-	A

Local

G	T	A	G	C	T	T	A	A	G	G	T	T	A
T	A	G	A	T	A	T	A	T	A	T	A	T	A

Local vs. Global Alignment (cont'd)



- Global Alignment

```

--T--CC-C-AGT--TATGT-CAGGGGACACG-A-GCATGCAGA-GAC
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
AATTGCCGCC-GTCGT-T-TTCAG----CA-GTTATG-T-CAGAT--C
  
```

- Local Alignment – better alignment to find conserved segment

```

          tccCAGTTATGTCAGgggacacgagcatgcagagac
          |||||
aattgccgccgtcggttttcagCAGTTATGTCAGatc
  
```

Local Alignments: Why?



- ❧ Two genes in different species may be similar over short conserved regions and dissimilar over remaining regions.
- ❧ Example:
 - ❧ **Homeobox genes** are a large group of similar **genes** that direct the formation of many body structures during early embryonic development
 - ❧ In humans, the **homeobox gene** group contains an estimated 235 functional **genes** and 65 pseudogenes
 - ❧ **Homeobox genes** have a short region called the *homeodomain* that is highly conserved between species.
- ❧ A global alignment ??? find the homeodomain

Can or
Cannot

Local vs. Global Alignment (cont'd)



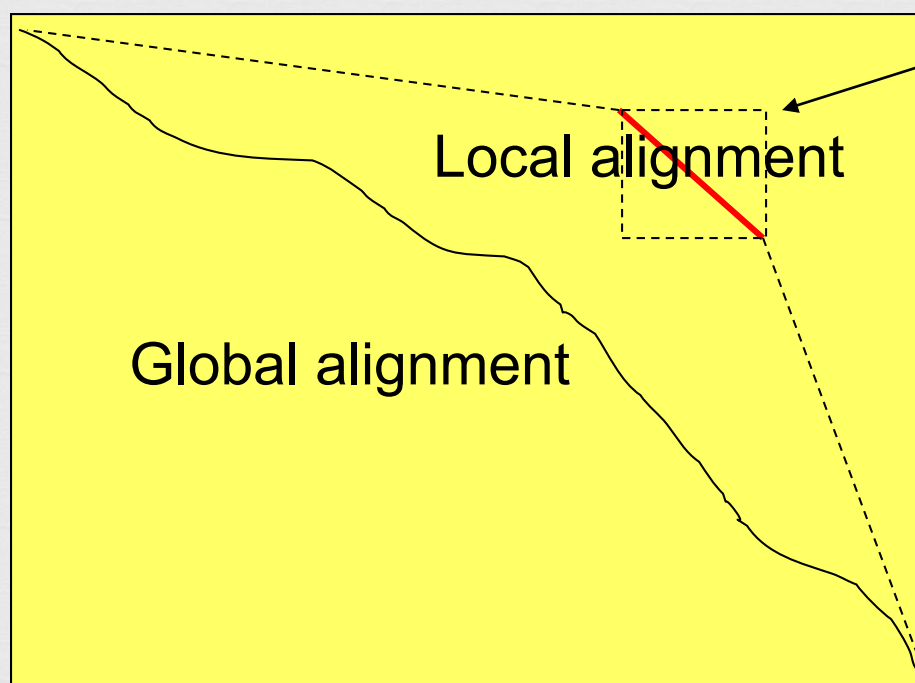
STOP and Think: A global alignment algorithm can perform local alignment as well.

True or False?

The Local Alignment Problem

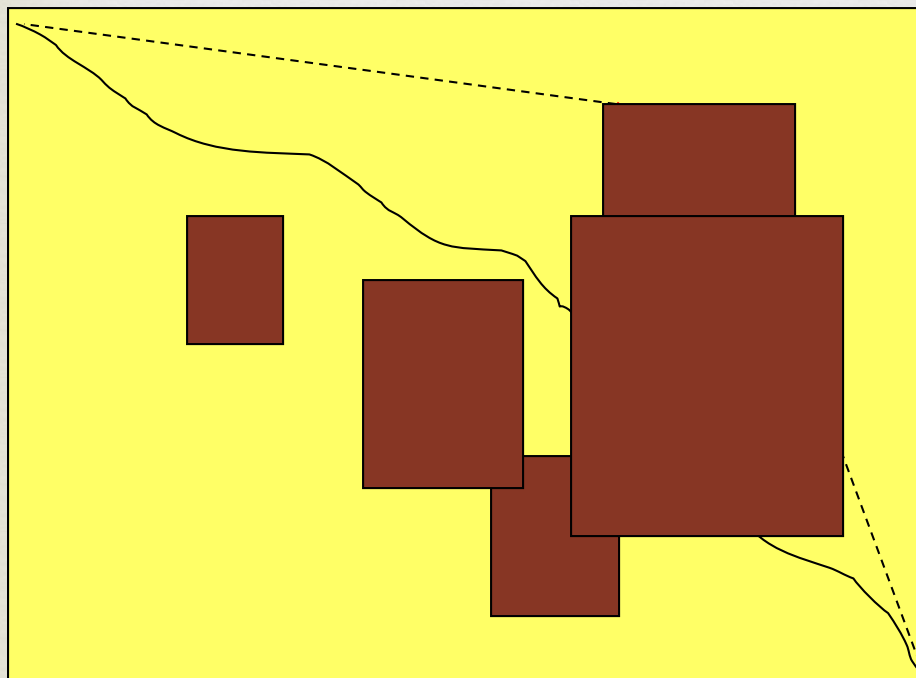
- ❧ Goal: Find the best local alignment between two strings
- ❧ Input : Strings v , w and scoring matrix δ
- ❧ Output : Alignment of substrings of v and w whose alignment score is maximum among all possible alignment of all possible substrings

Local Alignment: Example



Compute a “mini”
Global Alignment to
get Local

Local Alignment: Running Time



- Long run time $O(n^4)$:
 - In the grid of size $n \times n$ there are $\sim n^2$ vertices (i,j) that may serve as a source.
 - For each such vertex computing alignments from (i,j) to (i',j') takes $O(n^2)$ time.



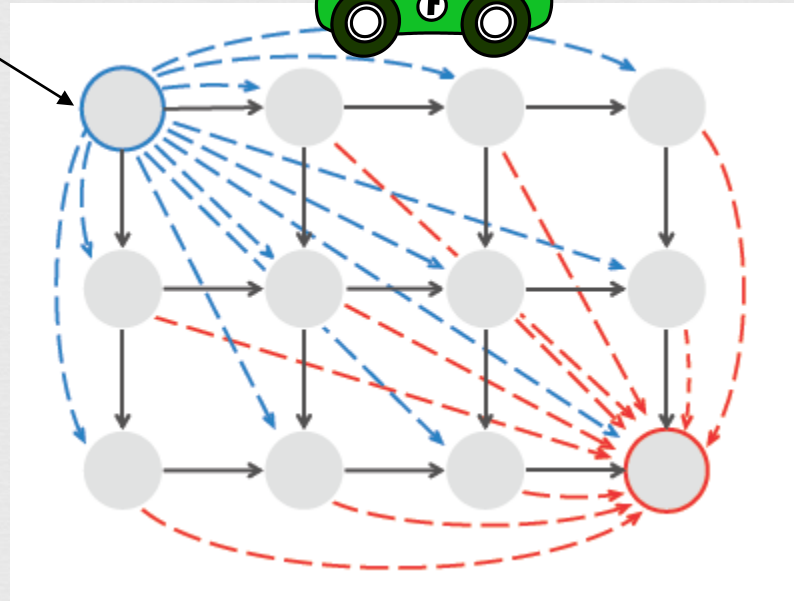
ANY SOLUTION?!

Local Alignment: Free Rides



Yeah, a free ride!

Vertex (0,0)



The dashed edges represent the free rides from (0,0) to every other node.

Local alignment

Global alignment initialization:

$$V(0, j) = g_j$$

$$V(i, 0) = g_i$$

Local alignment recurrence

$$V(0, j) = 0$$

$$V(i, 0) = 0$$

Local alignment

Global alignment recurrence:

$$V(i, j) = \max \left\{ \begin{array}{l} V(i-1, j-1) + S(x_i, y_j) \\ V(i-1, j) + g \\ V(i, j-1) + g \end{array} \right\}$$

Match/mismatch
delete
insert

Local alignment recurrence

$$V(i, j) = \max \left\{ \begin{array}{l} 0 \\ V(i-1, j-1) + S(x_i, y_j) \\ V(i-1, j) + g \\ V(i, j-1) + g \end{array} \right\}$$

Free ride

Local alignment traceback



- ✧ Let $T(i,j)$ be the traceback matrices and m and n be length of input sequences.
- ✧ Global alignment traceback:
 - ✧ Begin from $T(m,n)$ and stop at $T(0,0)$.
- ✧ Local alignment traceback:
 - ✧ Find i^*, j^* such that $T(i^*, j^*)$ is the maximum over all $T(i,j)$.
 - ✧ Begin traceback from $T(i^*, j^*)$ and stop when $T(i,j) \leq 0$.

Local Alignment: Free Rides

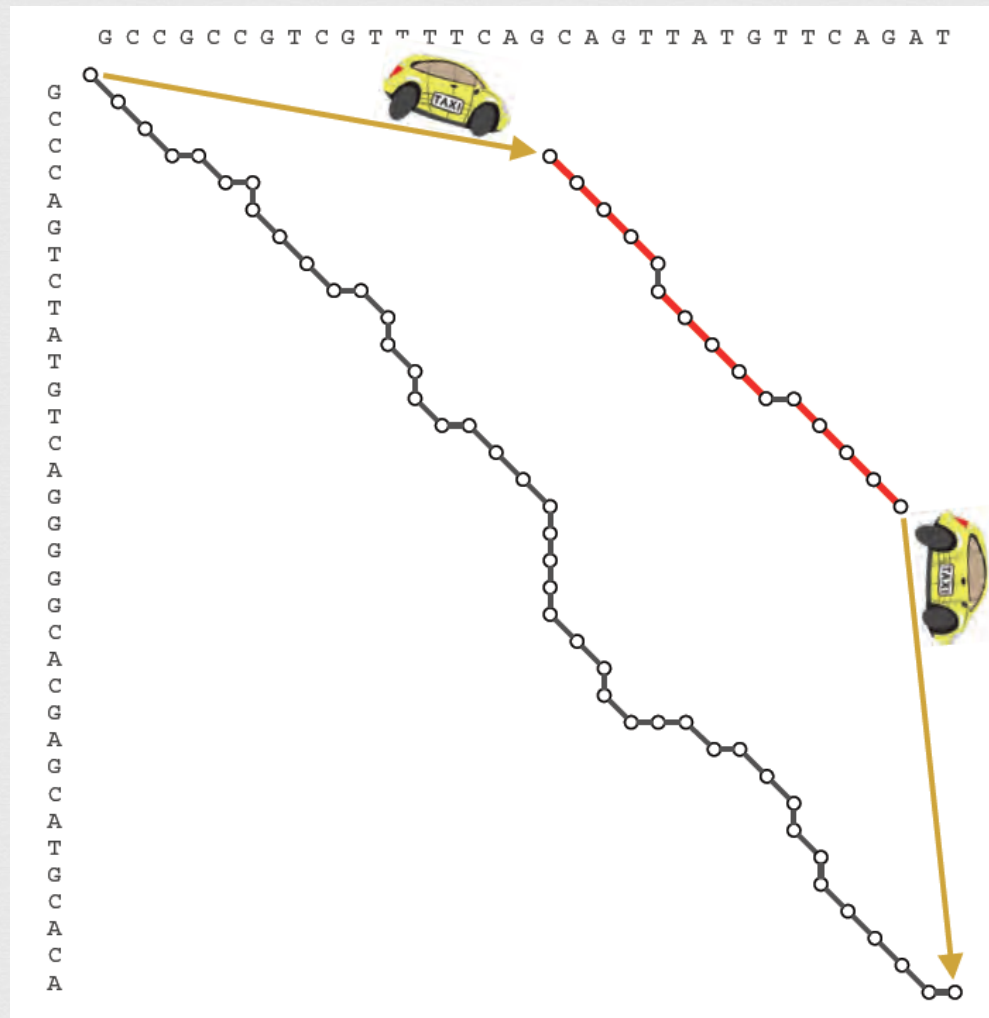


- Adding zero-weight edges from $(0, 0)$ to every node has made the source node $(0, 0)$ a predecessor of every node (i, j) .

$$s_{i,j} = \max \begin{cases} 0 \\ s_{i-1,j} + \text{Score}(v_i, -) \\ s_{i,j-1} + \text{Score}(-, w_j) \\ s_{i-1,j-1} + \text{Score}(v_i, w_j) \end{cases}$$

- Also, because node (n,m) now has every other node as a predecessor, $s_{n,m}$ will be equal to the largest value of $s_{i,j}$ over the entire alignment graph.

Sample Local Alignment



Outline



- Introduction to Sequence Alignment
- The Manhattan Tourist Problem
- Sequence Alignment is the Manhattan Tourist Problem in Disguise
- An Introduction to Dynamic Programming: The Manhattan Tourist Problem Revisited
- From Global to Local Alignment
- **Penalizing Insertions and Deletions in Sequence Alignment**

From LCS to Alignment: Change up the Scoring



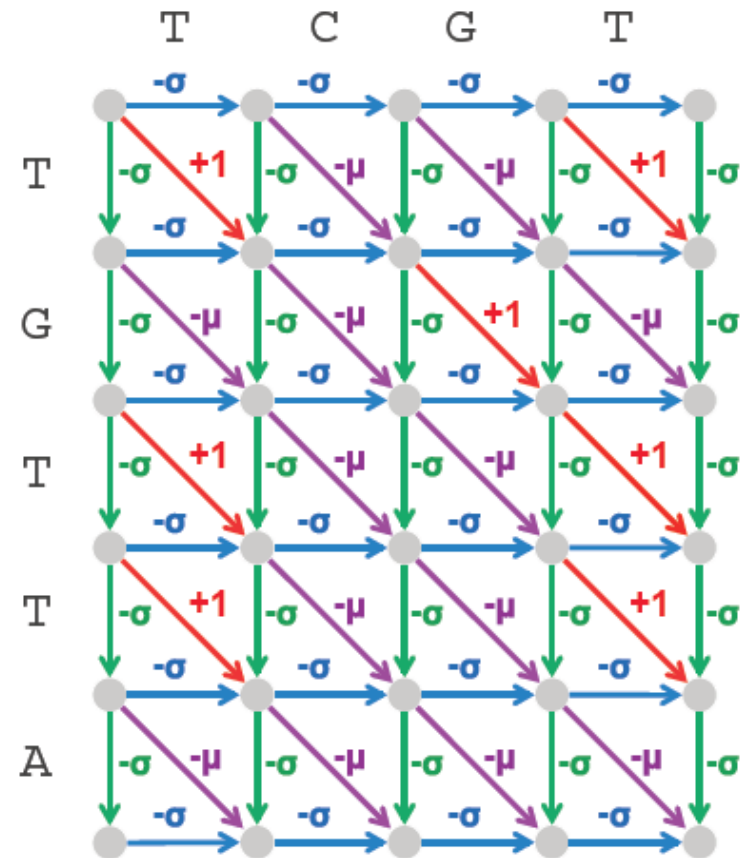
- ❧ The Longest Common Subsequence (LCS) problem – the simplest form of sequence alignment – allows only insertions and deletions (no mismatches).
- ❧ In the LCS Problem, we scored 1 for matches and 0 for indels
- ❧ Consider penalizing indels and mismatches with negative scores
- ❧ Simplest *scoring schema*:
 - +1 : match premium
 - μ : mismatch penalty
 - σ : indel penalty

Simple Scoring



When mismatches are penalized by $-\mu$, indels are penalized by $-\sigma$, and matches are rewarded with $+1$, the resulting score is:

$$\#matches - \mu(\#mismatches) - \sigma(\#indels)$$



The Global Alignment Problem

Find the best alignment between two strings under a given scoring schema

Input : Strings v and w and a scoring schema

Output : Alignment of maximum score

$$\begin{aligned} \uparrow \rightarrow &= -\sigma \\ \swarrow \begin{cases} = 1 & \text{if match} \\ = -\mu & \text{if mismatch} \end{cases} \end{aligned}$$

$$s_{i,j} = \max \begin{cases} s_{i-1,j-1} + 1 & \text{if } v_i = w_j \\ s_{i-1,j-1} - \mu & \text{if } v_i \neq w_j \\ s_{i-1,j} - \sigma \\ s_{i,j-1} - \sigma \end{cases}$$

μ : mismatch penalty
 σ : indel penalty

The Local Alignment Recurrence



- The largest value of $s_{i,j}$ over the whole edit graph is the score of the best local alignment.
- The recurrence:

$$s_{i,j} = \max \begin{cases} 0 \\ s_{i-1,j-1} + \delta(v_i, w_j) \\ s_{i-1,j} + \delta(v_i, -) \\ s_{i,j-1} + \delta(-, w_j) \end{cases}$$

“free ride” edge entering into every vertex

Scoring Matrices

To generalize scoring, consider a $(4+1) \times (4+1)$ **scoring matrix** δ .

In the case of an amino acid sequence alignment, the scoring matrix would be a $(20+1) \times (20+1)$ size. The addition of 1 is to include the score for comparison of a gap character “-”.

This will simplify the algorithm as follows:

$$s_{i,j} = \max \begin{cases} s_{i-1,j-1} + \delta(v_i, w_j) \\ s_{i-1,j} + \delta(v_i, -) \\ s_{i,j-1} + \delta(-, w_j) \end{cases}$$

Making a Scoring Matrix



- ❧ Scoring matrices are created based on biological evidence.
- ❧ Alignments can be thought of as two sequences that differ due to mutations.
- ❧ Some of these mutations have little effect on the protein's function, therefore some penalties, $\delta(v_i, w_j)$, will be less harsh than others.

Scoring Schemes and Weight Matrices



❧ Important Point

- All algorithms to compare protein sequences rely on some scheme to score the equivalencing of each 210 possible pairs.
- 190 different pairs + 20 identical pairs
- Higher scores for identical/similar amino acids (e.g. A,A or I, L)
- Lower scores to different character (e.g. I, D)

1. Identity Scoring

- Simplest Scoring scheme
- Score 1 for Identical pairs
- Score 0 for Non-Identical pairs
- Unable to detect similarity

Scoring Schemes and Weight Matrices



2. Chemical Similarity Scoring

- ❖ Similarity based on Physio-chemical properties
- ❖ MacLachlan 1972, Based on size, shape, charge and polar
- ❖ Score 0 for opposite (e.g. E & F) and 6 for identical character

3. Observed Substitutions or PAM matrices

- ❖ Based on Observed Substitutions
- ❖ Chicken and Egg problem
- ❖ Dayhoff group in 1977 align sequence manually
- ❖ Observed Substitutions or point mutation frequency
- ❖ MATRICES are PAM30, PAM250, PAM100 etc

4. BLOSUM- Matrix derived from Ungapped Alignment

- Derived from Local Alignment instead of Global
- Henikoff and Henikoff derived matrix from conserved blocks
- BLOSUM80, BLOSUM62, BLOSUM35

PAM



- ❧ **Point Accepted Mutation** (Dayhoff et al.)
- ❧ 1 PAM = PAM_1 = 1% average change of all amino acid positions
 - ❧ After 100 PAMs of evolution, not every residue will have changed
 - ❧ some residues may have mutated several times
 - ❧ some residues may have returned to their original state
 - ❧ some residues may not changed at all

PAM_x

⌘ PAM_x = PAM₁^x

⌘ PAM₂₅₀ = PAM₁²⁵⁰

⌘ PAM₂₅₀ is a widely used scoring matrix:

		Ala	Arg	Asn	Asp	Cys	Gln	Glu	Gly	His	Ile	Leu	Lys	...
		A	R	N	D	C	Q	E	G	H	I	L	K	...
Ala	A	13	6	9	9	5	8	9	12	6	8	6	7	...
Arg	R	3	17	4	3	2	5	3	2	6	3	2	9	
Asn	N	4	4	6	7	2	5	6	4	6	3	2	5	
Asp	D	5	4	8	11	1	7	10	5	6	3	2	5	
Cys	C	2	1	1	1	52	1	1	2	2	2	1	1	
Gln	Q	3	5	5	6	1	10	7	3	7	2	3	5	
...														
Trp	W	0	2	0	0	0	0	0	0	1	0	1	0	
Tyr	Y	1	1	2	1	3	1	1	1	3	2	2	1	
Val	V	7	4	4	4	4	4	4	4	5	4	15	10	

BLOSUM



- ❧ **B**locks **S**ubstitution **M**atrix
- ❧ Scores derived from *observations* of the frequencies of substitutions in blocks of local alignments in related proteins
- ❧ Matrix name indicates evolutionary distance
 - ❧ BLOSUM62 was created using sequences sharing no more than 62% identity

The Blosum50 Scoring Matrix



	A	R	N	D	C	Q	E	G	H	I	L	K	M	F	P	S	T	W	Y	V	B	Z	X	*
A	5	-2	-1	-2	-1	-1	-1	0	-2	-1	-2	-1	-1	-3	-1	1	0	-3	-2	0	-2	-1	-1	-5
R	-2	7	-1	-2	-4	1	0	-3	0	-4	-3	3	-2	-3	-3	-1	-1	-3	-1	-3	-1	0	-1	-5
N	-1	-1	7	2	-2	0	0	0	1	-3	-4	0	-2	-4	-2	1	0	-4	-2	-3	4	0	-1	-5
D	-2	-2	2	8	-4	0	2	-1	-1	-4	-4	-1	-4	-5	-1	0	-1	-5	-3	-4	5	1	-1	-5
C	-1	-4	-2	-4	13	-3	-3	-3	-3	-2	-2	-3	-2	-2	-4	-1	-1	-5	-3	-1	-3	-3	-2	-5
Q	-1	1	0	0	-3	7	2	-2	1	-3	-2	2	0	-4	-1	0	-1	-1	-1	-3	0	4	-1	-5
E	-1	0	0	2	-3	2	6	-3	0	-4	-3	1	-2	-3	-1	-1	-1	-3	-2	-3	1	5	-1	-5
G	0	-3	0	-1	-3	-2	-3	8	-2	-4	-4	-2	-3	-4	-2	0	-2	-3	-3	-4	-1	-2	-2	-5
H	-2	0	1	-1	-3	1	0	-2	10	-4	-3	0	-1	-1	-2	-1	-2	-3	2	-4	0	0	-1	-5
I	-1	-4	-3	-4	-2	-3	-4	-4	-4	5	2	-3	2	0	-3	-3	-1	-3	-1	4	-4	-3	-1	-5
L	-2	-3	-4	-4	-2	-2	-3	-4	-3	2	5	-3	3	1	-4	-3	-1	-2	-1	1	-4	-3	-1	-5
K	-1	3	0	-1	-3	2	1	-2	0	-3	-3	6	-2	-4	-1	0	-1	-3	-2	-3	0	1	-1	-5
M	-1	-2	-2	-4	-2	0	-2	-3	-1	2	3	-2	7	0	-3	-2	-1	-1	0	1	-3	-1	-1	-5
F	-3	-3	-4	-5	-2	-4	-3	-4	-1	0	1	-4	0	8	-4	-3	-2	1	4	-1	-4	-4	-2	-5
P	-1	-3	-2	-1	-4	-1	-1	-2	-2	-3	-4	-1	-3	-4	10	-1	-1	-4	-3	-3	-2	-1	-2	-5
S	1	-1	1	0	-1	0	-1	0	-1	-3	-3	0	-2	-3	-1	5	2	-4	-2	-2	0	0	-1	-5
T	0	-1	0	-1	-1	-1	-1	-2	-2	-1	-1	-1	-1	-2	-1	2	5	-3	-2	0	0	-1	0	-5
W	-3	-3	-4	-5	-5	-1	-3	-3	-3	-3	-2	-3	-1	1	-4	-4	-3	15	2	-3	-5	-2	-3	-5
Y	-2	-1	-2	-3	-3	-1	-2	-3	2	-1	-1	-2	0	4	-3	-2	-2	2	8	-1	-3	-2	-1	-5
V	0	-3	-3	-4	-1	-3	-3	-4	-4	4	1	-3	1	-1	-3	-2	0	-3	-1	5	-4	-3	-1	-5
B	-2	-1	4	5	-3	0	1	-1	0	-4	-4	0	-3	-4	-2	0	0	-5	-3	-4	5	2	-1	-5
Z	-1	0	0	1	-3	4	5	-2	0	-3	-3	1	-1	-4	-1	0	-1	-2	-2	-3	2	5	-1	-5
X	-1	-1	-1	-1	-2	-1	-1	-2	-1	-1	-1	-1	-1	-2	-2	-1	0	-3	-1	-1	-1	-1	-1	-5
*	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	1

Scoring Indels: Naive Approach



✧ A fixed penalty σ is given to every indel:

✧ $-\sigma$ for 1 indel,

✧ -2σ for 2 consecutive indels

✧ -3σ for 3 consecutive indels, etc.

Penalty for **100 consecutive indels** ????

Affine Gap Penalties

⌘ In nature, a series of k indels often come as a single event rather than a series of k single nucleotide events:

ATA__GC
ATATTGC

This is more likely.

ATAG_GC
AT_GTGC

This is less likely.

Normal scoring would
give the same score for
both alignments

Accounting for Gaps

✧ *Gaps*- contiguous sequence of spaces in one of the rows

✧ Score for a gap of length x is:

$$-(\sigma + \varepsilon (x-1))$$

○ where $\sigma > 0$ is the penalty for introducing a gap:

gap opening penalty

○ σ will be large relative to ε :

gap extension penalty

✧ because you do not want to add too much of a penalty for extending the gap.

Scoring Insertions and Deletions

match = 1
mismatch = 0

Total Score: 4

A	T	G	T	T	A	T	A
A	T	G	T	G	C	G	T A T A

Total Score (new): $8 - 3.2 = 4.8$

Total Score (old): $8 - 3 \times 3 = -1$

Gap parameters:

$\sigma = 3$ (gap opening)

$\varepsilon = 0.1$ (gap extension)

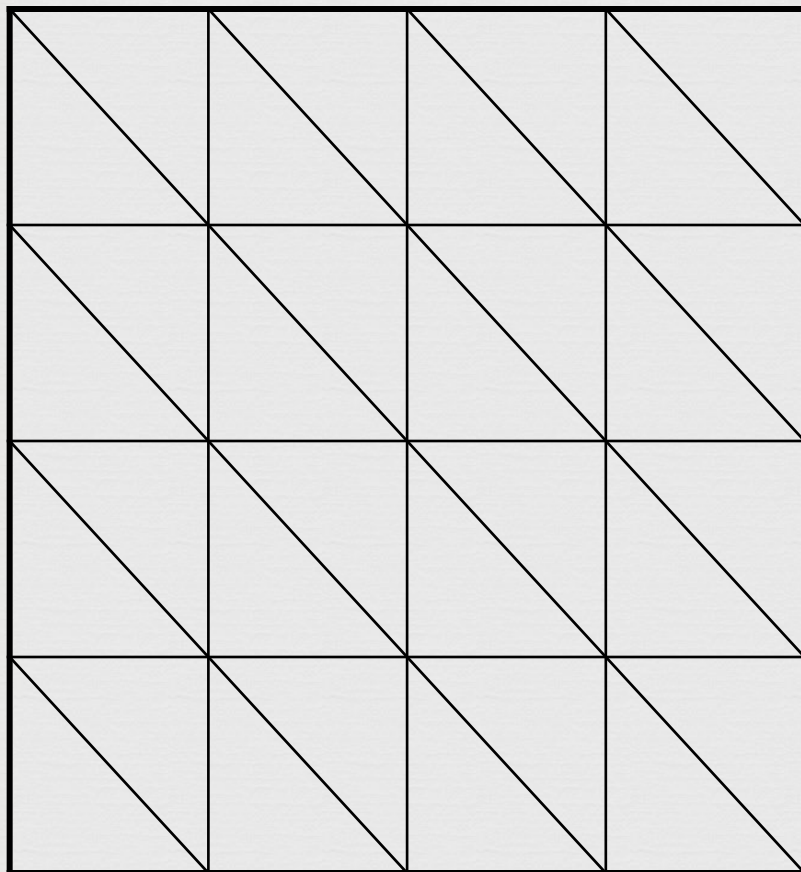
$x = 3$ (gap length)

$$\sum \text{penalty} = -3 - (3 - 1) 0.1 = -3.2$$

A	T	G	T	-	-	-	T	A	T	A
A	T	G	T	G	C	G	T	A	T	A

insertion / deletion

Affine Gap Penalties and Edit Graph

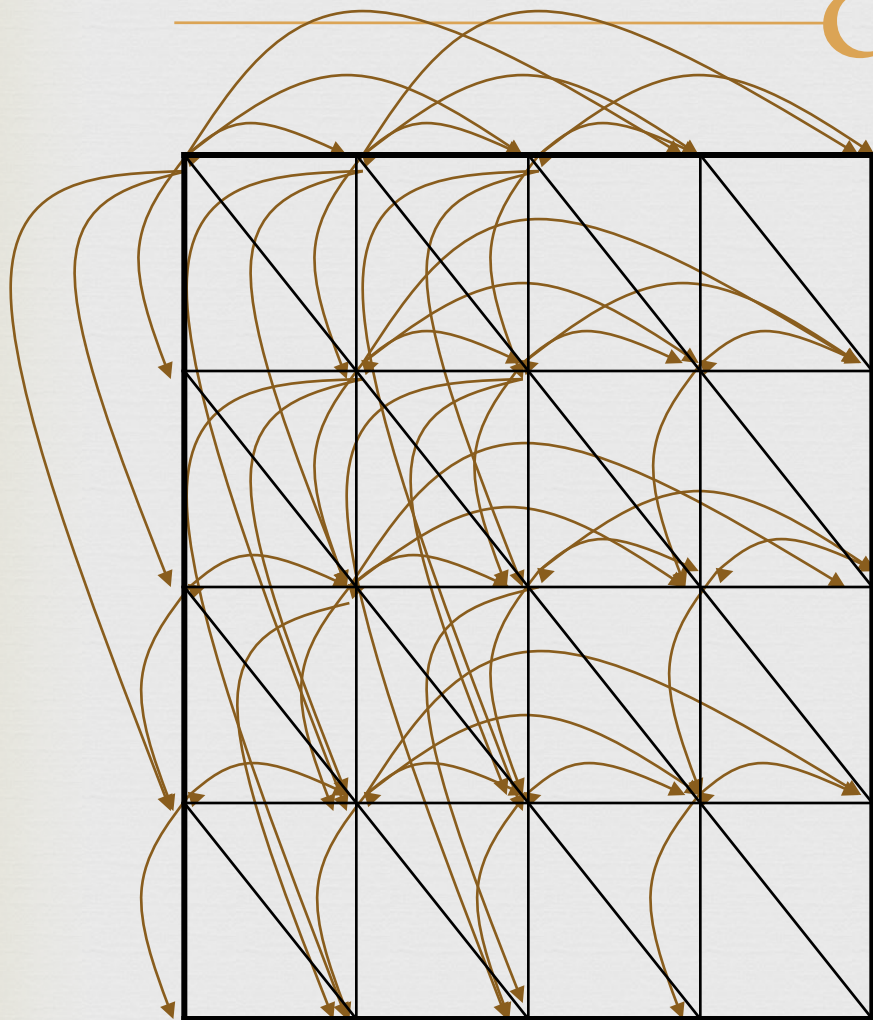


"How should this scoring method be considered?"

To reflect affine gap penalties we have to add “long” horizontal and vertical edges to the edit graph. Each such edge of length x should have weight

$$- \sigma - (x-1) * \varepsilon$$

Adding “Affine Penalty” Edges to the Edit Graph



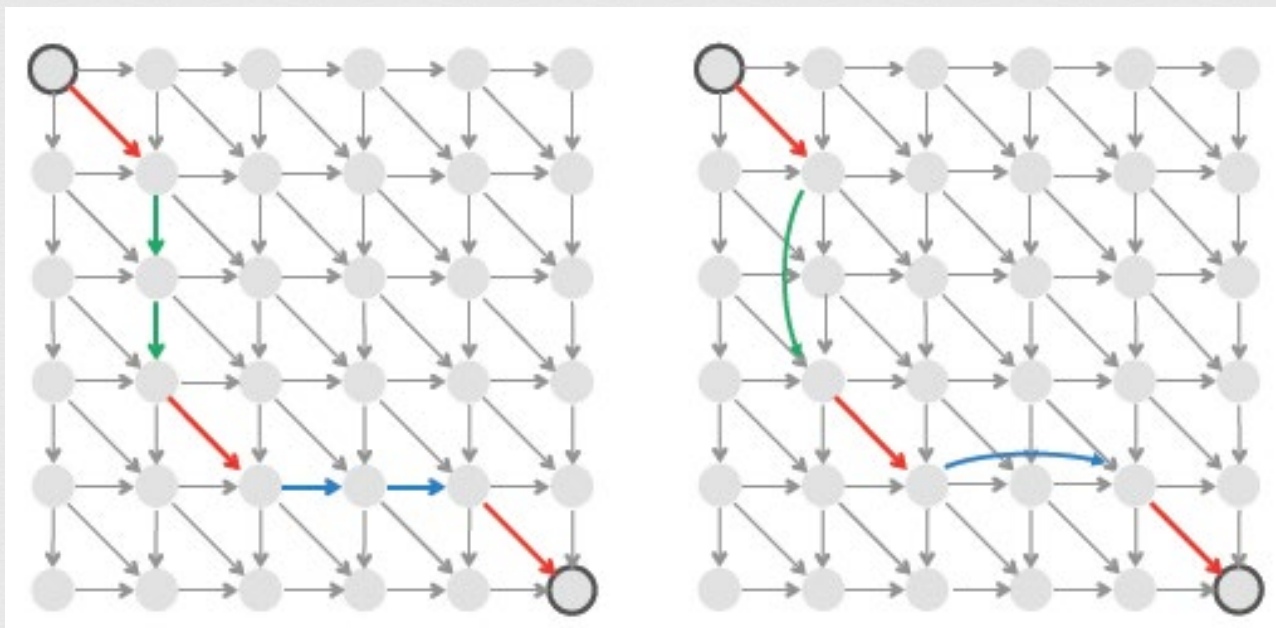
There are many such edges!

So the complexity increases
from $O(n^2)$ to $O(n^3)$



ANY SOLUTION?!

Different Gap Policies

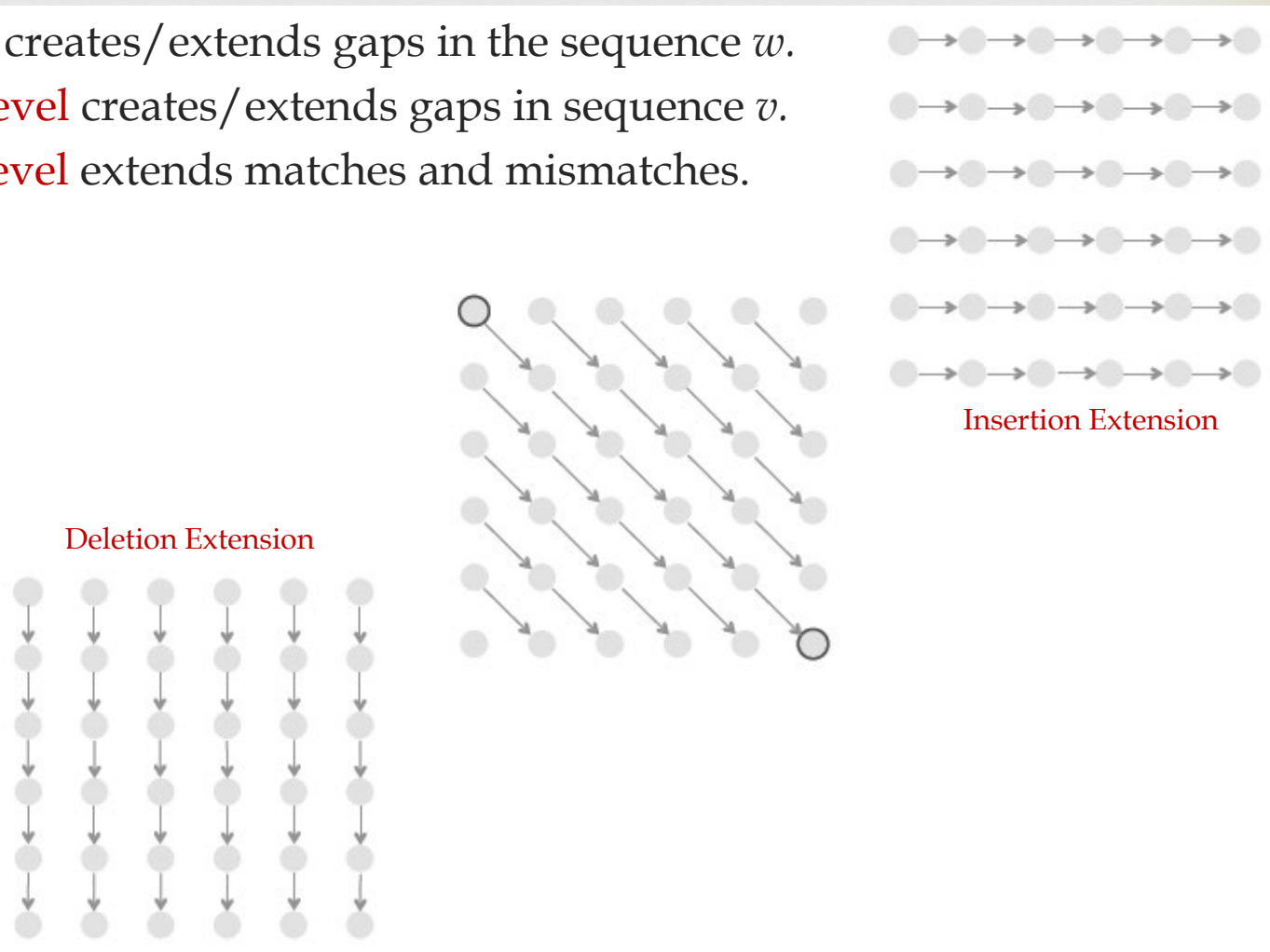


Solution: The trick to decreasing the number of edges in the DAG
 for the Alignment with Affine Gap Penalties Problem →
 to increase the number of nodes

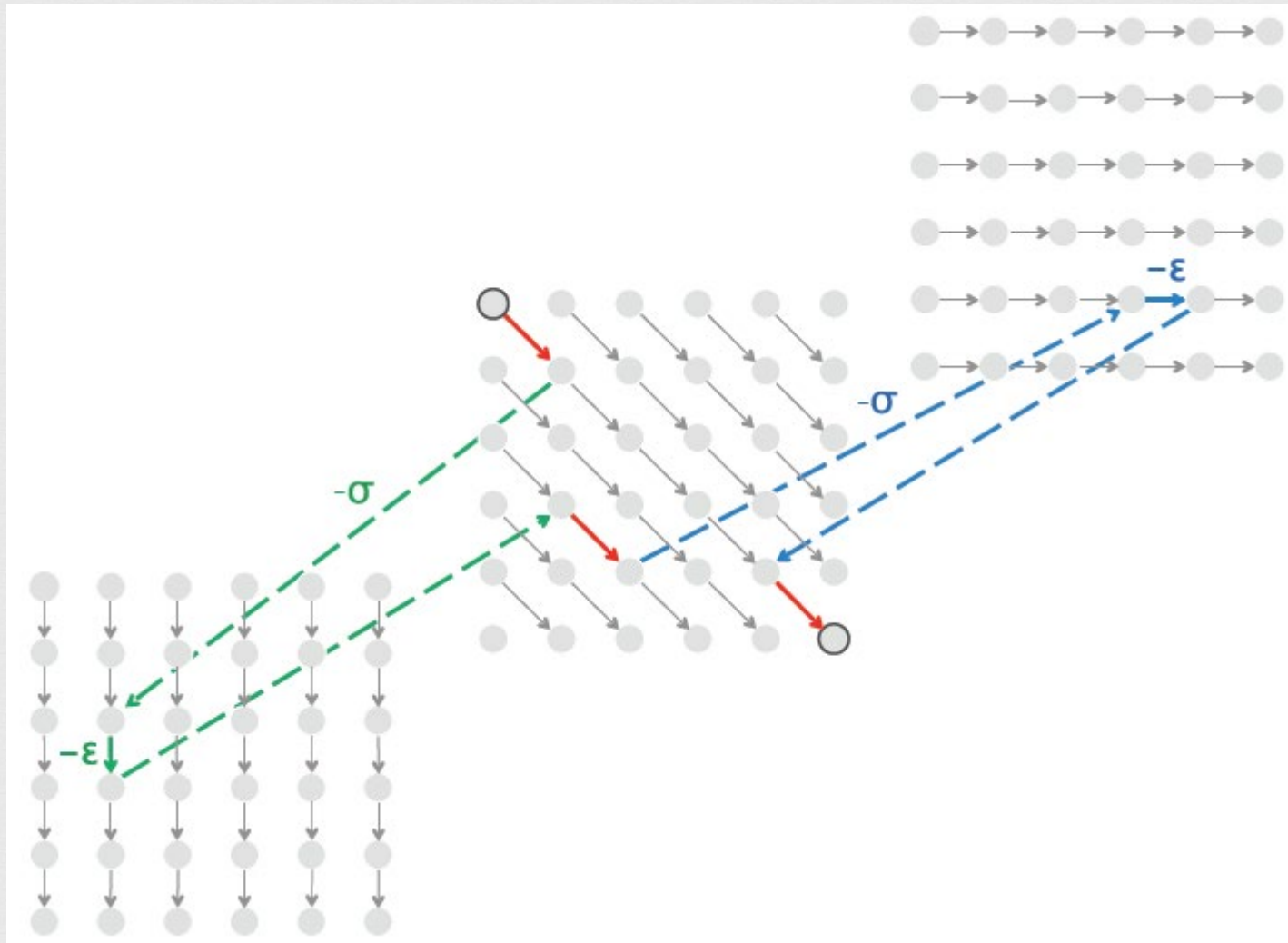
Affine Gap Penalties and 3 Layer Manhattan Grid



- ❧ The **top level** creates/extends gaps in the sequence w .
- ❧ The **bottom level** creates/extends gaps in sequence v .
- ❧ The **middle level** extends matches and mismatches.



Manhattan in 3 Layers



Switching between 3 Layers



- ⌘ - σ : Jumping penalty is assigned to moving from the main level to either the upper level or the lower level
- ⌘ - ϵ : Gap extension penalty for each continuation on a level other than the main level)

$$\begin{aligned}
 \text{lower}_{i,j} &= \max \begin{cases} \text{lower}_{i-1,j} - \epsilon \\ \text{middle}_{i-1,j} - \sigma \end{cases} \\
 \text{middle}_{i,j} &= \max \begin{cases} \text{lower}_{i,j} \\ \text{middle}_{i-1,j-1} + \text{Score}(v_i, w_j) \\ \text{upper}_{i,j} \end{cases} \\
 \text{upper}_{i,j} &= \max \begin{cases} \text{upper}_{i,j-1} - \epsilon \\ \text{middle}_{i,j-1} - \sigma \end{cases}
 \end{aligned}$$

Conclusion

❖ Techniques of Alignment

- Simple Alignment, Alignment with Gaps
- Dynamic Programming (DP) for Global Alignment (Needleman-Wunsch)
- Local Alignment (Smith-Waterman algorithm 1981)

❖ Scoring Schemes

- Identity Scoring
- Chemical Similarity Scoring
- Observed Substitution or PAM Matrices
- BLOSUM: Matrix Derived from Ungapped Alignment
- Gap Penalty (Opening, Extended)

References



- ❧ Phillip Compeau and Pavel Pevzner, *Bioinformatics Algorithms: An Active Learning Approach*, Chapter 5.
- ❧ Huang, X. (1994). On global sequence alignment. *Bioinformatics*, 10(3), 227-235.
- ❧ Mott, R. (1999). Local sequence alignments with monotonic gap penalties. *Bioinformatics (Oxford, England)*, 15(6), 455-462.